

**PERANCANGAN DAN EVALUASI MEKANISME
SINKRONISASI DIREKTORI INKREMENTAL
BERBASIS OPERASI *FILESYSTEM*
MENGUNAKAN eBPF**

Laporan Tugas Akhir

Disusun sebagai syarat kelulusan tingkat sarjana

Oleh

Panji Sri Kuncara Wisma

NIM: 13522028



**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Agustus 2026

**PERANCANGAN DAN EVALUASI MEKANISME
SINKRONISASI DIREKTORI INKREMENTAL
BERBASIS OPERASI *FILESYSTEM*
MENGUNAKAN eBPF**

Laporan Tugas Akhir

Oleh

Panji Sri Kuncara Wisma

NIM: 13522028

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Telah disetujui sebagai draft Laporan Tugas Akhir
untuk menjadi syarat mengikuti Seminar TA

Bandung, 26 Agustus 2026

Mengetahui

Pembimbing,

Achmad Imam Kistijantoro, S.T., M.Sc., Ph.D.

NIP. 19730809 200604 1 001

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa:

1. Pengerjaan dan penulisan Laporan Tugas Akhir ini dilakukan tanpa menggunakan bantuan yang tidak dibenarkan.
2. Segala bentuk kutipan dan acuan terhadap tulisan orang lain yang digunakan di dalam penyusunan laporan tugas akhir ini telah dituliskan dengan baik dan benar.
3. Laporan Tugas Akhir ini belum pernah diajukan pada program pendidikan di perguruan tinggi mana pun.

Jika terbukti melanggar hal-hal di atas, saya bersedia dikenakan sanksi sesuai dengan Peraturan Akademik dan Kemahasiswaan Institut Teknologi Bandung bagian Penegakan Norma Akademik dan Kemahasiswaan khususnya Pasal 2.1 dan Pasal 2.2.

Bandung, 26 Agustus 2026

Panji Sri Kuncara Wisma

NIM 13522028

ABSTRAK

PERANCANGAN DAN EVALUASI MEKANISME SINKRONISASI DIREKTORI INKREMENTAL BERBASIS OPERASI *FILESYSTEM* MENGGUNAKAN eBPF

Oleh:

Panji Sri Kuncara Wisma
NIM: 13522028

Banyak aplikasi menyimpan *state* persisten dalam bentuk berkas serta struktur direktori pada *filesystem*. Karena *state* tersebut termaterialisasi sebagai direktori, menyediakan replika direktori pada mesin lain dapat dipakai untuk ketahanan terhadap kegagalan tanpa bergantung pada logika internal aplikasi. Untuk kebutuhan tersebut, mekanisme yang dicari harus inkremental, tidak bergantung pada aplikasi maupun jenis *filesystem* tertentu, dapat digunakan antara dua mesin, serta menghasilkan artefak perubahan yang dapat dipindahkan. *rsync* dipilih sebagai acuan pembandingan karena memenuhi karakteristik tersebut dalam implementasi yang matang. *rsync* menentukan perubahan melalui pemindaian serta perbandingan kondisi dua direktori. Cara tersebut membuat biaya penentuan perubahan sebanding dengan ukuran data yang sudah ada, bukan dengan besarnya perubahan yang benar-benar terjadi.

Penelitian ini merancang serta mengimplementasikan mekanisme alternatif yang menangkap perubahan tepat ketika operasi *filesystem* berlangsung menggunakan eBPF pada lapisan *Virtual File System*. Mekanisme tersebut merekam operasi yang mengubah isi maupun struktur direktori beserta isi tulisannya, menyusunnya menjadi artefak yang dapat dipindahkan, lalu menerapkannya pada direktori replika di mesin lain. Evaluasi dilakukan pada dua mesin fisik yang terpisah dengan *rsync* sebagai acuan pembandingan.

Hasil pengukuran menunjukkan biaya sinkronisasi *rsync* tumbuh dari 75,8 milidetik menjadi 1.988,2 milidetik ketika ukuran berkas bertambah dari 10 MB menjadi 1000 MB, sedangkan biaya mekanisme yang dibangun tetap pada kisaran 10 milidetik. Sebaliknya, mekanisme tersebut kalah ketika rentang *byte* yang sama ditulis berulang sebelum sinkronisasi, dengan titik potong pada sekitar dua ribu penulisan. Biaya yang dibebankan pada jalur eksekusi aplikasi terukur sebesar 3,0 mikrodetik per operasi tulis serta 12,7 mikrodetik per permintaan HTTP pada layanan berbasis SQLite. Pengujian pada SQLite, PostgreSQL, serta MySQL menunjukkan mekanisme tersebut bekerja tanpa modifikasi aplikasi maupun pemindahan direktori data.

Kata kunci: *eBPF, Virtual File System, replikasi direktori, rsync, penangkapan perubahan*

ABSTRACT

DESIGN AND EVALUATION OF AN INCREMENTAL DIRECTORY SYNCHRONIZATION MECHANISM BASED ON FILESYSTEM OPERATIONS USING eBPF

By:

Panji Sri Kuncara Wisma

NIM: 13522028

Many applications keep their persistent state as files and directory structures on the filesystem. Because that state is materialised as a directory, providing a directory replica on another machine can serve fault tolerance without depending on any application's internal logic. For this purpose, the desired mechanism must be incremental, independent of the application and of a particular filesystem type, usable between two machines, and able to produce a transferable change artifact. `rsync` is selected as the comparison baseline because it satisfies these characteristics in a mature implementation. `rsync` determines changes by scanning and comparing the state of two directories. That approach makes the cost of determining changes proportional to the size of the existing data rather than to the size of the changes that actually occurred.

This research designs and implements an alternative mechanism that captures changes at the moment filesystem operations occur, using eBPF at the Virtual File System layer. The mechanism records the operations that alter directory content or structure together with the bytes they write, assembles them into a transferable artefact, and then applies that artefact to a replica directory on another machine. The evaluation was carried out on two separate physical machines with `rsync` as the comparison baseline.

The measurements show that the synchronisation cost of `rsync` grows from 75.8 milliseconds to 1,988.2 milliseconds as the file size increases from 10 MB to 1000 MB, while the cost of the proposed mechanism stays at around 10 milliseconds. Conversely, the proposed mechanism loses when the same byte range is rewritten repeatedly before synchronisation, with a crossover point at roughly two thousand writes. The cost imposed on the application execution path measures 3.0 microseconds per write operation and 12.7 microseconds per HTTP request on an SQLite-backed service. Tests on SQLite, PostgreSQL, and MySQL show that the mechanism works without modifying the application or relocating its data directory.

Keywords: eBPF, Virtual File System, directory replication, rsync, change capture

KATA PENGANTAR

Bandung, 26 Agustus 2026

Panji Sri Kuncara Wisma

DAFTAR ISI

ABSTRAK	iv
ABSTRACT	v
KATA PENGANTAR	vi
DAFTAR ISI	vii
DAFTAR GAMBAR	ix
DAFTAR TABEL	x
BAB I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	2
I.3 Batasan Masalah	3
I.4 Metodologi	4
BAB II KAJIAN PUSTAKA	6
II.1 Sinkronisasi <i>State</i> pada Direktori	6
II.1.1 <i>State</i> Persisten pada Direktori Aplikasi	6
II.1.2 Replikasi pada Level Aplikasi	6
II.1.3 Sinkronisasi Direktori Antarmesin	7
II.2 Pendekatan Penentuan Perubahan Direktori	7
II.2.1 Kriteria Pemilihan Acuan Pembanding	7
II.2.2 <i>rsync</i> sebagai Acuan Pembanding	8
II.2.3 Alternatif Penentuan Perubahan	9
II.3 Landasan Penangkapan Operasi <i>Filesystem</i>	10
II.3.1 Arsitektur <i>Virtual File System</i> Linux	10
II.3.2 <i>Extended Berkeley Packet Filter</i> (eBPF)	11
II.4 <i>Gap</i> dan Posisi Riset	12
BAB III ANALISIS PERSOALAN DAN RANCANGAN SOLUSI	13
III.1 Analisis	13
III.1.1 Analisis Permasalahan	13
III.1.2 Kebutuhan Sistem	14
III.1.3 Alternatif Solusi	16
III.2 Rancangan	19

III.2.1 Rancangan Struktural	19
III.2.2 Arsitektur Sistem.....	21
III.2.3 Alur Penentuan Perubahan	22
III.2.4 Rancangan Penangkap Perubahan Berbasis eBPF.....	23
BAB IV IMPLEMENTASI DAN EVALUASI	25
IV.1 Lingkungan.....	25
IV.2 Implementasi	26
IV.2.1 Batasan Implementasi.....	26
IV.2.2 Kakas yang Digunakan	27
IV.2.3 Implementasi Penangkap Perubahan Berbasis eBPF.....	27
IV.2.4 Implementasi Program Penerap Perubahan Data	33
IV.2.5 Implementasi rsync sebagai Acuan Pembanding	34
IV.2.6 Implementasi Orkestrator Eksperimen	35
IV.3 Evaluasi.....	36
IV.3.1 Konfigurasi Pengujian	36
IV.3.2 Pengujian Sistem	37
IV.3.3 Analisis Hasil	39
BAB V PENUTUP.....	46
V.1 Kesimpulan.....	46
V.2 Saran.....	47
DAFTAR PUSTAKA	49

DAFTAR GAMBAR

Gambar II.1. VFS Linux (Linux Kernel Teaching, 2025)	10
Gambar III.1. Susunan komponen sistem eksperimen	20
Gambar III.2. Arsitektur sistem eksperimen pada dua mesin	22
Gambar III.3. Perbandingan alur penentuan perubahan pada kedua meka- nisme	23
Gambar III.4. Struktur penangkap perubahan berbasis eBPF	24
Gambar IV.1. Ilustrasi pembatasan lingkup perekaman beserta penyusunan <i>path</i>	30
Gambar IV.2. Struktur orkestrator eksperimen	35
Gambar IV.3. Perintah untuk menjalankan eksperimen pertama	36
Gambar IV.4. Perintah untuk menjalankan eksperimen kedua	37
Gambar IV.5. Perintah untuk menjalankan eksperimen ketiga beserta eks- perimen keempat	37
Gambar IV.6. Biaya menghasilkan satu artefak perubahan terhadap ukuran berkas yang sudah ada, dengan volume perubahan tetap sebesar 4 KB	41
Gambar IV.7. Biaya sinkronisasi ketika rentang <i>byte</i> 4 KB yang sama ditulis ulang sebelum sinkronisasi dijalankan	41
Gambar IV.8. Distribusi latensi satu penulisan berukuran 4 KB dengan pe- nangkapan mati dibandingkan hidup	43
Gambar IV.9. Distribusi latensi satu permintaan HTTP tulis pada aplikasi daftar tugas di atas SQLite dengan penangkapan mati dibandingkan hidup	43

DAFTAR TABEL

Tabel II.1.	Kriteria pemilihan acuan pembanding	8
Tabel III.1.	Kebutuhan Fungsional	14
Tabel III.2.	Kebutuhan Non-Fungsional.....	15
Tabel III.3.	Perbandingan lapisan penangkapan perubahan	17
Tabel III.4.	Perbandingan mekanisme instrumentasi kernel.....	19
Tabel IV.1.	Lingkungan implementasi dan pengukuran	25
Tabel IV.2.	Titik penautan program penangkap pada lapisan VFS.....	31
Tabel IV.3.	Hasil pengujian pada aplikasi nyata.....	39
Tabel IV.4.	Variabel yang digunakan pada setiap eksperimen.....	40

BAB I

PENDAHULUAN

I.1 Latar Belakang

Banyak aplikasi dan basis data menyimpan *state* persisten sebagai berkas dan struktur direktori pada *filesystem*. *Filesystem Hierarchy Standard*, misalnya, menetapkan `/var/lib/` sebagai lokasi untuk informasi *state* yang dipertahankan antar-invokasi dan umumnya tetap berlaku setelah *reboot* (Linux Foundation, 2015, Sec. 5.8). PostgreSQL juga menyimpan seluruh data dari sebuah *database cluster* di bawah satu direktori data (PostgreSQL Global Development Group, 2025a, Sec. 18.2). Karena *state* tersebut termaterialisasi sebagai direktori, menyediakan replika direktori pada mesin lain dapat mendukung *backup*, pemulihan, serta ketahanan terhadap kegagalan tanpa bergantung pada logika internal aplikasi.

Beberapa mekanisme dapat digunakan untuk menyediakan replika tersebut. Replikasi melalui log internal aplikasi efektif untuk aplikasi yang memang menyediakannya, tetapi terikat pada logika dan format penyimpanan aplikasi tersebut. Replikasi pada level perangkat blok berada di bawah lapisan *filesystem* sehingga tidak mengenal batas satu direktori (LINBIT, 2020). Pengiriman *snapshot* secara inkremental dapat dilakukan pada *filesystem* tertentu, tetapi tidak bersifat umum lintas *filesystem*. Sementara itu, antarmuka notifikasi seperti `inotify` hanya menunjukkan objek yang berubah, bukan bagian *byte* yang berubah, sehingga isi perubahan tetap perlu diperoleh melalui pembacaan dan perbandingan (Linux man-pages project, 2026).

Agar dapat digunakan secara umum, mekanisme sinkronisasi yang dicari perlu bekerja secara inkremental, tidak bergantung pada jenis aplikasi, relatif tidak bergantung pada jenis *filesystem*, dapat digunakan antara dua mesin, menghasilkan artefak perubahan yang dapat dipindahkan, serta tersedia dalam implementasi yang matang. `rsync` dipilih sebagai acuan pembandingan karena memenuhi seluruh karakteristik tersebut (rsync contributors, 2026; Tridgell & Mackerras, 1996). Dengan demikian, pemilihan `rsync` dalam penelitian ini diturunkan dari karakteristik mekanisme

yang diperlukan, bukan dari klaim mengenai tingkat penggunaannya.

rsync menentukan perubahan dengan merekonstruksinya dari *state* kedua direktori. Berkas kandidat mula-mula ditentukan melalui pemeriksaan ukuran dan waktu modifikasi, kemudian bagian yang berbeda dicari melalui pencocokan *checksum* blok antara sumber dan replika (rsync contributors, 2026; Tridgell & Mackerras, 1996). Pendekatan ini memiliki tiga konsekuensi. Pertama, biaya penentuan perubahan dipengaruhi oleh ukuran pohon direktori, bukan hanya oleh besarnya perubahan yang benar-benar terjadi. Kedua, direktori replika perlu dipelihara sebagai salinan pembanding. Ketiga, setiap sinkronisasi memerlukan pemindaian; karena itu, peningkatan frekuensi sinkronisasi untuk memperoleh replika yang lebih mutakhir juga meningkatkan biaya total.

Penelitian ini mengeksplorasi pendekatan lain, yaitu menentukan perubahan dengan menangkapnya saat operasi *filesystem* berlangsung. Isi penulisan hanya tersedia di dalam kernel pada saat operasi terjadi. Oleh karena itu, penangkapan isi perubahan perlu dilakukan pada lapisan kernel. Lapisan *Virtual File System* (VFS) menyediakan operasi berkas yang seragam lintas *filesystem* (Linux kernel developers, 2026a). Program *extended Berkeley Packet Filter* (eBPF) dapat ditautkan pada fungsi di lapisan tersebut dan diperiksa oleh *verifier* sebelum dimuat, sehingga penangkapan dapat dilakukan tanpa mengubah kode sumber kernel, memuat modul kernel tambahan, maupun memindahkan direktori *state* aplikasi (Sharaf dkk., 2022; Witzke dkk., 2024).

Mekanisme yang dibangun merekam operasi yang mengubah isi atau struktur direktori beserta isi tulisannya, menyusunnya menjadi artefak perubahan, lalu menerapkannya pada direktori replika di mesin lain. Berbeda dari pemindaian, pendekatan ini tidak memiliki mekanisme koreksi dari pengamatan *state* akhir. Satu operasi yang terlewat dapat membuat replika tidak setara secara permanen. Karena itu, penelitian ini tidak hanya mengukur biaya penangkapan perubahan, tetapi juga menguji kelengkapan penangkapan dan kesetaraan replika yang dihasilkannya.

I.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, penelitian ini membahas perancangan dan evaluasi mekanisme sinkronisasi direktori inkremental yang menentukan perubahan

dari operasi *filesystem*, bukan dari pemindaian *state*. Rumusan masalah penelitian adalah sebagai berikut.

1. Bagaimana merancang mekanisme berbasis eBPF pada lapisan VFS untuk menangkap, menyusun, dan menerapkan perubahan direktori secara inkremental antarmesin tanpa memindai pohon direktori atau membaca ulang isi direktori sumber?
2. Apakah perekaman operasi *filesystem* yang dilakukan, dalam ruang lingkup operasi yang ditangani, cukup lengkap untuk menghasilkan replika yang setara dengan direktori sumber?
3. Bagaimana biaya mekanisme berbasis eBPF dibandingkan dengan *rsync* pada variasi ukuran *state* dan jumlah penulisan sebelum sinkronisasi, serta berapa tambahan latensi yang dibebankan pada jalur eksekusi aplikasi?

I.3 Batasan Masalah

Penelitian ini membatasi ruang lingkup kajian pada mekanisme sinkronisasi inkremental satu arah dari direktori sumber ke direktori replika. Batasan yang diterapkan adalah sebagai berikut.

1. Lingkungan Kajian.

Direktori sumber dan direktori replika ditempatkan pada dua mesin fisik yang terpisah. Kedua direktori berada pada penyimpanan lokal masing-masing mesin, dan artefak perubahan dikirim melalui jaringan. Implementasi sumber dijalankan pada Linux dengan hak akses *root*, informasi tipe BTF, serta dukungan penautan pada fungsi internal kernel.

2. Ruang Lingkup Data.

Objek yang direplikasi adalah berkas reguler dan direktori. Entri *filesystem* khusus, termasuk *symbolic link*, *hard link*, *device file*, serta *Unix domain socket*, tidak termasuk dalam cakupan penelitian.

3. Kesetaraan dan Metadata.

Kesetaraan antara sumber dan replika didefinisikan berdasarkan kesamaan struktur direktori dan isi *byte* berkas. Izin akses, kepemilikan, *timestamp*, atribut tambahan, dan ACL tidak menjadi kriteria kesetaraan. Penelitian ini juga tidak mengevaluasi jaminan durabilitas pada replika.

4. **Operasi yang Ditangani.**

Mekanisme menangani operasi mutasi terhadap berkas reguler dan direktori, yaitu pembuatan berkas, penulisan biasa, penulisan vektor, penulisan asinkron *native*, pemotongan berkas, alokasi ruang berkas yang didukung, serta pembuatan, penghapusan, dan penamaan ulang entri direktori. Operasi yang hanya mengubah metadata, seperti `chmod`, `chown`, dan `utimensat`, berada di luar cakupan penelitian.

5. **Model Konsistensi dan Kelengkapan.**

Sistem menargetkan konsistensi pada akhirnya: replika setara dengan sumber setelah seluruh artefak perubahan diterapkan dan tidak ada perubahan baru. Sistem tidak menjamin batas maksimum ketertinggalan replika. Jika kehilangan rekaman terdeteksi, mekanisme menolak menerbitkan artefak perubahan; penelitian ini tidak merancang pemulihan berbasis pemindaian untuk kondisi tersebut.

6. **Pembanding dan Fokus Evaluasi.**

`rsync` digunakan sebagai satu-satunya acuan pembanding yang menentukan perubahan melalui pemindaian dan perbandingan *state*. Evaluasi berfokus pada kesetaraan replika, biaya sinkronisasi, ukuran artefak, serta tambahan latensi pada operasi aplikasi.

I.4 Metodologi

Tugas akhir ini dilaksanakan melalui tahapan kerja sebagai berikut.

1. **Identifikasi Permasalahan dan Studi Literatur.**

Tahap ini menetapkan karakteristik mekanisme sinkronisasi direktori, menganalisis konsekuensi penentuan perubahan berbasis pemindaian, serta menelaah lapisan VFS dan eBPF sebagai tempat penangkapan operasi.

2. **Analisis dan Perancangan Sistem.**

Hasil identifikasi digunakan untuk merancang penangkap perubahan berbasis eBPF, program pemuat, format artefak perubahan, penerap perubahan pada mesin replika, `rsync` sebagai acuan pembanding, serta orkestrator eksperimen. Rancangan juga menetapkan penolakan artefak apabila perekaman tidak lengkap.

3. Implementasi Sistem.

Tahap ini mengimplementasikan komponen yang dirancang untuk menangkap operasi mutasi pada lapisan VFS, menyusun perubahan menjadi artefak inkremental, mengirimkannya ke mesin replika, dan menerapkannya kembali tanpa membaca direktori sumber.

4. Pengujian Fungsional.

Pengujian memverifikasi kesetaraan direktori sumber dan replika pada setiap titik ukur. Pengujian juga dijalankan pada SQLite, PostgreSQL, dan MySQL untuk memeriksa bahwa mekanisme dapat digunakan tanpa modifikasi aplikasi maupun pemindahan direktori data.

5. Eksperimen dan Analisis Biaya.

Eksperimen membandingkan biaya sinkronisasi mekanisme berbasis eBPF dan `rsync` pada variasi ukuran *state* dan jumlah penulisan berulang sebelum sinkronisasi. Eksperimen terpisah mengukur tambahan latensi pada operasi tulis berkas dan permintaan HTTP aplikasi yang menggunakan SQLite.

6. Analisis dan Evaluasi.

Hasil pengujian dianalisis untuk menilai kelengkapan penangkapan, kesetaraan replika, serta kondisi beban kerja ketika pendekatan berbasis operasi lebih atau kurang menguntungkan dibandingkan pendekatan berbasis pemindaian.

BAB II

KAJIAN PUSTAKA

Bab ini membahas konsep dan penelitian terdahulu yang mendasari sinkronisasi direktori inkremental. Pembahasan dimulai dari *state* persisten pada direktori aplikasi, dilanjutkan dengan cara menentukan perubahan pada direktori, lalu ditutup dengan landasan penangkapan operasi pada lapisan *Virtual File System* (VFS).

II.1 Sinkronisasi *State* pada Direktori

II.1.1 *State* Persisten pada Direktori Aplikasi

State persisten adalah bagian dari kondisi aplikasi yang disimpan pada media non-volatil agar dapat digunakan kembali setelah proses aplikasi berhenti atau mesin di-*reboot*. Dalam penelitian ini, pembahasan dibatasi pada *state* yang termaterialisasi sebagai berkas reguler dan struktur direktori. Filesystem Hierarchy Standard menetapkan `/var/lib/` sebagai lokasi untuk informasi *state* yang dipertahankan antar-invokasi dan umumnya tetap berlaku setelah *reboot* (Linux Foundation, 2015, Sec. 5.8). PostgreSQL juga menyimpan seluruh data sebuah *database cluster* di bawah satu direktori data (PostgreSQL Global Development Group, 2025a, Sec. 18.2).

Karakteristik tersebut membuat replikasi pada level direktori relevan untuk aplikasi yang berbeda. Replika tidak perlu memahami model data, protokol, maupun logika internal aplikasi selama seluruh *state* yang diperlukan berada di bawah direktori yang direplikasi. Kesetaraan yang menjadi sasaran penelitian ini karena itu adalah kesamaan struktur direktori dan isi *byte* berkas, bukan kesamaan semantik pada satu jenis aplikasi tertentu.

II.1.2 Replikasi pada Level Aplikasi

Replikasi pada level aplikasi memanfaatkan pengetahuan aplikasi mengenai perubahan *state*-nya. PostgreSQL, misalnya, dapat mempertahankan *standby* melalui *write-ahead log* (WAL). *Standby* menerima catatan WAL dari sumber lalu menja-

lankan proses pemulihan untuk mengejar keadaan sumber (PostgreSQL Global Development Group, 2025b). MySQL mengirim perubahan melalui *binary log* yang direkam sebagai rangkaian *event* dan dieksekusi kembali oleh replika (Oracle Corporation, 2025).

Pendekatan tersebut tepat ketika aplikasi menyediakan mekanisme replikasi dan administrator dapat menggunakannya. Akan tetapi, unit perubahan, format log, cara pemulihan, serta prasyarat konsistensinya berbeda antar aplikasi. Oleh karena itu, pendekatan ini tidak dapat langsung digunakan sebagai mekanisme sinkronisasi generik untuk sebuah direktori *state* tanpa asumsi terhadap aplikasi yang menulis di dalamnya.

II.1.3 Sinkronisasi Direktori Antarmesin

Sinkronisasi direktori adalah upaya menjaga agar isi sebuah direktori replika pada mesin lain pada akhirnya setara dengan direktori sumber. Pada level direktori, operasi seperti pembuatan, penghapusan, penamaan ulang serta penulisan berkas perlu dipropagasikan agar struktur serta isi berkas pada kedua sisi kembali sama. Kebutuhan ini merupakan penerapan sinkronisasi data pada objek *filesystem* (IBM, 2025).

Dalam penelitian ini, sumber berperan sebagai pihak yang menghasilkan perubahan dan replika berperan sebagai pihak yang menerapkannya. Model ini bersifat satu arah dan menargetkan konsistensi pada akhirnya. Model tersebut cukup untuk memisahkan persoalan penentuan perubahan dari persoalan resolusi konflik yang muncul pada sinkronisasi dua arah atau banyak penulis.

II.2 Pendekatan Penentuan Perubahan Direktori

II.2.1 Kriteria Pemilihan Acuan Pemanding

Penelitian ini mencari pendekatan alternatif untuk menentukan perubahan pada direktori. Karena itu, acuan pemanding perlu dipilih dari karakteristik masalah, bukan dari tingkat penggunaan suatu alat. Kriteria yang digunakan disajikan pada Tabel II.1.

Tabel II.1: Kriteria pemilihan acuan pembanding

Kriteria	Peran dalam penelitian
Inkremental	Hanya perubahan sejak keadaan sebelumnya yang perlu ditentukan dan dipindahkan. Kriteria ini membedakan sinkronisasi dari penyalinan penuh seperti <i>scp</i> atau <i>tar</i> .
Generik terhadap aplikasi	Mekanisme tidak boleh bergantung pada format data, log, maupun protokol replikasi sebuah aplikasi.
Relatif tidak bergantung pada <i>filesystem</i>	Mekanisme perlu bekerja pada direktori biasa tanpa menuntut satu jenis <i>filesystem</i> atau fitur penyimpanan khusus.
Antarmesin	Hasil penentuan perubahan harus dapat digunakan untuk memperbarui direktori pada mesin lain.
Artefak perubahan dapat dipindahkan	Perubahan perlu dapat disalurkan ke mesin replika dalam bentuk aliran atau artefak yang tidak menuntut pembacaan kembali direktori sumber ketika diterapkan.

Tidak ada satu kriteria yang cukup dengan sendirinya. Replikasi pada level aplikasi, misalnya, dapat inkremental dan antarmesin, tetapi tidak generik terhadap aplikasi. Sebaliknya, penyalinan penuh dapat bersifat generik, tetapi tidak membedakan perubahan dari seluruh isi direktori. Berdasarkan gabungan kriteria tersebut, *rsync* dipilih sebagai acuan pembanding. Pemilihan itu tidak didasarkan pada tingkat penggunaan *rsync*. *rsync* dipilih karena memenuhi karakteristik yang dibutuhkan penelitian.

II.2.2 *rsync* sebagai Acuan Pembanding

rsync adalah alat penyalinan berkas yang dapat bekerja secara lokal maupun antara dua *host* melalui *remote shell* atau *rsync daemon* (rsync contributors, 2026). Alat ini memenuhi kebutuhan sinkronisasi direktori inkremental dan tidak bergantung pada logika aplikasi maupun jenis *filesystem* tertentu. Hasil penentuan perubahannya disalurkan ke sisi tujuan melalui protokol transfer, sehingga direktori replika dapat diperbarui di mesin lain.

Cara *rsync* memperoleh perubahan berbeda dari mekanisme yang dibangun dalam penelitian ini. Pada konfigurasi bawaan, *rsync* melakukan *quick check* untuk memilih berkas kandidat berdasarkan ukuran dan waktu modifikasi antara sumber dan tujuan (rsync contributors, 2026). Untuk berkas kandidat yang perlu diperbarui, *delta-transfer algorithm* dapat menentukan bagian yang berbeda dengan mencocokkan *checksum* blok. Sisi penerima membentuk daftar *checksum* blok dari berkas yang telah ada, sedangkan sisi pengirim memindai berkas sumber untuk mencari

padanan dan mengirimkan data literal atau referensi blok yang sesuai (Tridgell & Mackerras, 1996).

Pendekatan tersebut merekonstruksi perubahan dari *state* yang diamati ketika sinkronisasi dijalankan. Akibatnya, setiap sinkronisasi tetap menelusuri pohon direktori. Berkas kandidat perlu dibaca untuk menentukan bagian yang berbeda. Biaya penentuan perubahan dengan demikian dipengaruhi oleh ukuran *state* yang harus diamati, bukan hanya oleh volume perubahan yang baru terjadi. Direktori replika juga berperan sebagai salinan pembanding yang harus tersedia di sisi tujuan. Sebaliknya, pengamatan *state* pada setiap siklus memberikan kesempatan untuk menyelaraskan kembali perbedaan yang terlihat pada siklus tersebut.

II.2.3 Alternatif Penentuan Perubahan

Beberapa mekanisme lain dapat memenuhi sebagian dari kriteria pada Tabel II.1, tetapi tidak seluruhnya. Mekanisme tersebut tetap penting untuk menjelaskan sumber informasi perubahan yang tersedia dan alasan penelitian ini mencari pendekatan lain.

Antarmuka notifikasi *filesystem* seperti *inotify* menyediakan rekaman yang berisi penanda pengamatan, jenis peristiwa, *cookie*, serta nama entri yang terkena peristiwa (Linux man-pages project, 2026). Rekaman tersebut menunjukkan objek yang berubah, tetapi tidak membawa rentang *byte* atau isi yang ditulis. Notifikasi dapat mempersempit daftar berkas yang perlu diperiksa, namun isi perubahan masih perlu diperoleh dengan membaca kembali dan membandingkan berkas sumber. Tanpa pemindaian penuh sebagai pemulihan, notifikasi yang hilang dapat membuat perubahan tidak tertangani.

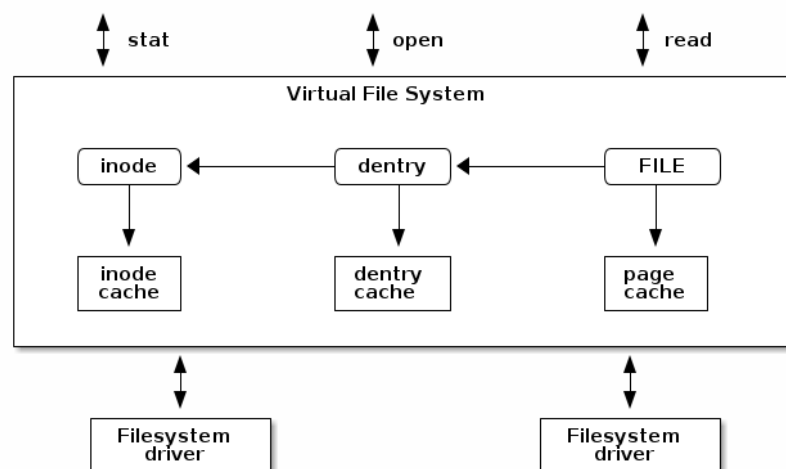
Snapshot pada *copy-on-write filesystem* menyediakan alternatif yang lebih dekat dengan artefak perubahan. Sebagai contoh, *btrfs send* dapat menghasilkan aliran lengkap atau inkremental dari sebuah subvolume *read-only*. Mode inkremental membentuk perbedaan relatif terhadap *snapshot* yang menjadi acuan. *btrfs receive* lalu membangun kembali subvolume pada sisi tujuan (Btrfs developers, 2026). Pendekatan ini efisien untuk lingkungan Btrfs, tetapi bergantung pada subvolume, *snapshot*, serta alat *btrfs* pada *filesystem* tersebut. Karena itu, pendekatan ini tidak memenuhi kebutuhan untuk bekerja pada direktori biasa lintas *filesystem*.

Distributed Replicated Block Device (DRBD) bekerja pada lapisan perangkat blok. Penulisan pada perangkat virtual direplikasi ke perangkat pada mesin lain (LINBIT, 2020). Karena berada di bawah *filesystem*, DRBD tidak mengenal nama berkas, struktur direktori, maupun batas direktori target. DRBD berguna untuk mereplikasi perangkat penyimpanan, tetapi tidak menyediakan granularitas direktori yang dibutuhkan penelitian ini.

II.3 Landasan Penangkapan Operasi *Filesystem*

II.3.1 Arsitektur *Virtual File System* Linux

Virtual File System (VFS) adalah lapisan di kernel Linux yang menyediakan antarmuka *filesystem* bagi program *userspace* sekaligus abstraksi internal agar berbagai implementasi *filesystem* dapat hidup berdampingan (Linux kernel developers, 2026a). Aplikasi tetap menggunakan operasi seperti `open(2)` dan `write(2)` tanpa perlu mengetahui apakah objek berada pada ext4, XFS, NFS, atau *filesystem* lain.



Gambar II.1: VFS Linux (Linux Kernel Teaching, 2025)

VFS memodelkan *filesystem* dengan beberapa objek inti. *Superblock* mewakili sebuah *filesystem* yang dipasang, *inode* mewakili objek *filesystem*, *dentry* menghubungkan nama dengan *inode*, serta objek *file* mewakili berkas yang dibuka oleh proses. Ketika sebuah proses melakukan operasi, VFS menyelesaikan *path* menjadi objek tersebut lalu meneruskan operasi ke implementasi *filesystem* yang sesuai (Linux kernel developers, 2026a).

Posisi tersebut penting bagi penelitian ini. Di atas VFS, operasi masih dekat dengan antarmuka aplikasi dan memiliki banyak variasi *system call*. Di bawah VFS, penangkapan mulai terikat pada satu implementasi *filesystem*. Pada lapisan perangkat blok, berkas dan direktori tidak lagi dikenal. VFS menjadi titik yang mempertahankan objek serta operasi *filesystem* generik sekaligus tetap berlaku lintas *filesystem*.

II.3.2 *Extended Berkeley Packet Filter (eBPF)*

Extended Berkeley Packet Filter (eBPF) memungkinkan program yang ditentukan pengguna dijalankan di dalam kernel Linux. Sebelum dimuat, program diperiksa oleh *verifier* untuk membatasi akses memori dan memastikan perilaku eksekusinya sesuai aturan eBPF (Sharaf dkk., 2022). Dengan demikian, eBPF dapat digunakan untuk instrumentasi kernel tanpa mengubah kode sumber kernel, memuat modul kernel tambahan, maupun *me-reboot* sistem.

Untuk program penelusuran, kernel menyediakan penautan *fentry* dan *fexit* pada awal dan akhir fungsi (Linux kernel developers, 2026b). Penautan pada kedua titik memungkinkan program memperoleh argumen ketika operasi dimulai dan nilai balik ketika operasi selesai. Kemampuan tersebut relevan untuk membedakan operasi yang berhasil dari operasi yang gagal, serta untuk menyusun rekaman perubahan pada lapisan VFS.

Witzke dkk. menunjukkan penggunaan eBPF pada fungsi VFS untuk memantau I/O tingkat rendah pada *scientific workflow*. Program mereka ditautkan pada awal dan akhir fungsi VFS agar argumen serta nilai balik operasi dapat diamati (Witzke dkk., 2024). Studi tersebut menunjukkan kelayakan VFS sebagai titik instrumentasi yang generik terhadap *filesystem*. Namun, tujuan utamanya adalah pemantauan dan pengaitan I/O dengan tugas aplikasi, bukan menyusun artefak perubahan yang dapat dipindahkan, menerapkannya pada replika, atau menguji kesetaraan direktori setelah penerapan.

Penautan pada fungsi VFS tetap memiliki konsekuensi. Fungsi yang ditautkan merupakan fungsi internal kernel yang dapat berubah antarversi. Tidak semua jalur I/O juga melewati satu fungsi yang sama. Karena itu, pemilihan titik penautan serta pengujian kelengkapan penangkapan menjadi bagian penting dari rancangan dan

evaluasi penelitian ini.

II.4 *Gap* dan Posisi Riset

Kajian pustaka menunjukkan bahwa mekanisme sinkronisasi dapat memperoleh informasi perubahan dari log aplikasi, *snapshot filesystem*, notifikasi *filesystem*, atau pengamatan *state* direktori. Untuk sinkronisasi direktori yang generik terhadap aplikasi, pendekatan berbasis *state* menjadi penting karena tidak memerlukan pengetahuan mengenai format data aplikasi. Pendekatan ini menentukan perubahan dengan memindai pohon direktori serta membandingkan *state* beserta isi berkas pada direktori sumber dengan direktori replika.

rsync merepresentasikan pendekatan berbasis *state* tersebut. Alat ini memenuhi kebutuhan sinkronisasi direktori yang inkremental, generik terhadap aplikasi, relatif tidak bergantung pada jenis *filesystem*, dapat digunakan antarmesin, serta dapat menyalurkan perubahan ke replika. Oleh karena itu, *rsync* digunakan sebagai acuan pembanding dalam penelitian ini. Namun, *rsync* menentukan perubahan dengan menelusuri kembali pohon direktori serta membaca berkas kandidat ketika sinkronisasi dijalankan. Biaya penentuan perubahan tersebut dipengaruhi oleh ukuran *state* yang diamati, bukan hanya oleh volume perubahan baru.

Gap penelitian ini terletak pada pencarian pendekatan lain yang memenuhi kriteria sinkronisasi direktori seperti *rsync* tanpa melakukan pemindaian ulang. eBPF berpotensi memberikan kontribusi melalui mekanisme yang langsung menangkap operasi VFS beserta isi perubahannya ketika operasi berlangsung. Rekaman operasi tersebut dapat disusun menjadi artefak perubahan inkremental untuk dipindahkan serta diterapkan pada direktori replika di mesin lain. Penelitian ini merancang dan mengevaluasi mekanisme berbasis eBPF untuk menilai apakah pendekatan tersebut dapat menghasilkan replika yang setara dengan biaya yang sesuai.

BAB III

ANALISIS PERSOALAN DAN RANCANGAN SOLUSI

Tujuan utama penulisan bab ini adalah untuk menjelaskan proses analisis dan perancangan pendekatan alternatif dalam menentukan perubahan yang terjadi pada sebuah direktori serta menyinkronkannya ke direktori di mesin lain. Pendekatan tersebut dirancang sedemikian rupa sehingga tidak bergantung pada jenis aplikasi maupun jenis *filesystem*, sebagaimana halnya yang dilakukan oleh *rsync*, tetapi tanpa perlu dilakukan pemindaian ulang pada seluruh pohon direktori.

III.1 Analisis

Bagian ini menganalisis permasalahan yang akan diselesaikan. Hasil analisis digunakan untuk merumuskan kebutuhan sistem. Selanjutnya, beberapa alternatif solusi dibandingkan untuk menentukan pendekatan yang sesuai.

III.1.1 Analisis Permasalahan

Berdasarkan latar belakang dan studi literatur yang telah dipaparkan, salah satu mekanisme sinkronisasi perubahan direktori yang bersifat generik adalah pemindaian dan perbandingan *state*, sebagaimana yang dilakukan oleh *rsync*. Cara ini menimbulkan dua konsekuensi. Pertama, biaya penentuan perubahan sebanding dengan ukuran pohon direktori dan bukan dengan besarnya perubahan yang benar-benar terjadi. Kedua, penentuan perubahan menuntut pembacaan ulang isi berkas kandidat pada kedua sisi. Oleh karena biaya tersebut dibayarkan setiap kali sinkronisasi dijalankan, pengaruh frekuensi sinkronisasi terhadap biaya total menjadi salah satu hal yang perlu diperhitungkan.

Pendekatan yang tidak merekonstruksi perubahan dari *state* setelah perubahan terjadi layak dicari karena informasi mengenai perubahan seharusnya sudah tersedia pada saat operasi *filesystem* berlangsung. Apabila informasi-informasi penting meng-

enai direktori dapat diperoleh, maka pemindaian pohon direktori serta pembacaan ulang isi berkas sumber tidak lagi diperlukan.

Berdasarkan analisis permasalahan dan pendekatan yang telah diuraikan, dirumuskan kebutuhan perangkat lunak sebagai berikut:

1. Sistem harus dapat menentukan perubahan pada sebuah direktori tanpa memindai pohon direktori dan tanpa membaca ulang isi berkas sumber.
2. Sistem harus dapat menerapkan perubahan yang telah ditangkap pada direktori replika di mesin lain.
3. Sistem harus menyediakan mekanisme pemindaian *state* sebagai acuan untuk mengevaluasi mekanisme yang dibangun.
4. Kedua mekanisme harus dapat dijalankan pada jenis direktori dan beban kerja yang sama untuk memperoleh data yang representatif.

Kebutuhan nomor 1 dan 2 merupakan kebutuhan terhadap mekanisme yang dibangun, sedangkan kebutuhan nomor 3 dan 4 merupakan kebutuhan terhadap lingkungan yang mengukurnya. Penjelasan lebih detail mengenai kebutuhan sistem terdapat pada bagian III.1.2.

III.1.2 Kebutuhan Sistem

Hasil analisis permasalahan pada bagian III.1.1 digunakan sebagai dasar untuk merumuskan kebutuhan fungsional dan nonfungsional sistem. Kebutuhan tersebut selanjutnya diterjemahkan ke dalam komponen-komponen yang akan diimplementasikan pada sistem eksperimen.

III.1.2.1 Kebutuhan Fungsional

Sistem eksperimen yang dikembangkan perlu memenuhi sejumlah kebutuhan fungsional. Kebutuhan tersebut dirumuskan berdasarkan hasil analisis permasalahan pada bagian III.1.1 dan disajikan pada Tabel III.1.

Tabel III.1: Kebutuhan Fungsional

ID	Deskripsi Kebutuhan
F-1	Sistem harus dapat merekam operasi mutasi, yaitu operasi yang mengubah isi maupun struktur direktori, beserta isi perubahan yang dihasilkannya

Tabel III.1: Kebutuhan Fungsional (lanjutan)

ID	Deskripsi Kebutuhan
F-2	Sistem harus dapat membatasi perekaman hanya pada direktori yang ditentukan, termasuk subdirektori yang dibuat ketika sistem sedang berjalan
F-3	Sistem harus dapat menyusun perubahan yang terekam menjadi sebuah artefak yang dapat dipindahkan ke mesin lain
F-4	Sistem harus dapat mengambil perubahan secara inkremental, yaitu hanya perubahan yang terjadi sejak pengambilan sebelumnya
F-5	Sistem harus dapat menerapkan artefak perubahan pada direktori replika tanpa memerlukan akses ke direktori sumber
F-6	Sistem harus menyediakan mekanisme pembandingan yang menentukan perubahan melalui pemindaian dan perbandingan <i>state</i> serta menghasilkan artefak yang sejenis
F-7	Sistem harus dapat memvariasikan ukuran data, kerapatan perubahan, serta kondisi penangkapan secara terkendali
F-8	Sistem harus dapat mencatat biaya penentuan perubahan, ukuran artefak, serta latensi operasi aplikasi
F-9	Sistem harus dapat memverifikasi kesetaraan antara direktori sumber dan direktori replika
F-10	Sistem harus dapat menjalankan eksperimen secara berulang dan otomatis untuk memperoleh data yang representatif

III.1.2.2 Kebutuhan Non-fungsional

Dalam pengembangannya, sistem eksperimen ini harus memenuhi beberapa kebutuhan non-fungsional. Kebutuhan non-fungsional ini diturunkan dari bagian I.2 dan analisis permasalahan pada bagian III.1.1. Kebutuhan non-fungsional sistem dapat dilihat pada tabel III.2.

Tabel III.2: Kebutuhan Non-Fungsional

ID	Deskripsi Kebutuhan
NF-1	Replika yang dihasilkan harus setara dengan direktori sumber setelah seluruh perubahan diterapkan
NF-2	Sistem tidak boleh menuntut penggunaan jenis <i>filesystem</i> tertentu, modifikasi aplikasi, maupun pemindahan direktori <i>state</i> aplikasi
NF-3	Sistem tidak boleh memerlukan pembacaan ulang isi direktori sumber untuk menentukan perubahan
NF-4	Biaya yang dibebankan sistem pada jalur eksekusi aplikasi harus dapat diisolasi dan diukur
NF-5	Kedua mekanisme yang dibandingkan harus dijalankan atas direktori, beban kerja, dan definisi kesetaraan yang sama

Kebutuhan NF-4 dinyatakan sebagai keterukuran dan bukan sebagai batas nilai tertentu, karena besar biaya pada jalur eksekusi aplikasi merupakan besaran yang hendak diukur pada penelitian ini.

III.1.3 Alternatif Solusi

Berdasarkan permasalahan pada bagian III.1.1 dan kebutuhan pada bagian III.1.2, terdapat berbagai macam alternatif solusi untuk menyelesaikan permasalahan tersebut. Pada penelitian ini, solusi yang dipilih adalah membangun mekanisme yang memiliki tujuan serupa dengan *rsync*, yaitu menjadikan direktori pada mesin tujuan identik dengan direktori sumber secara inkremental. Perbedaannya terletak pada cara memperoleh informasi perubahan. *rsync* merekonstruksi perubahan berdasarkan kondisi atau *state* terakhir sebuah direktori, sedangkan penelitian ini mencoba menangkap perubahan melalui sumber lain. Bagian ini akan membahas alternatif solusi yang dipertimbangkan, perbandingan antarsolusi, serta alasan pemilihan solusi yang digunakan dalam penelitian ini.

III.1.3.1 Pendekatan Penentuan Perubahan

Terdapat beberapa solusi yang dipertimbangkan dalam menentukan perubahan yang terjadi pada sebuah direktori, yaitu

1. Pemindaian dan perbandingan *state* direktori

Pendekatan ini menelusuri seluruh pohon direktori pada setiap sinkronisasi, menyaring berkas kandidat melalui pemeriksaan ukuran dan waktu modifikasi, lalu menentukan bagian berkas yang berbeda melalui pencocokan *checksum* blok antara kedua sisi, sebagaimana dijelaskan pada bagian II.2.2. Kelebihan pendekatan ini adalah perubahan tidak mungkin hilang secara permanen karena yang diamati adalah keadaan akhir direktori, sehingga perubahan yang tidak tertangani pada satu siklus tetap terlihat pada siklus berikutnya. Akan tetapi, biaya penentuan perubahan ditentukan oleh ukuran pohon direktori dan bukan oleh besarnya perubahan. Pendekatan ini juga menuntut pembacaan ulang isi berkas kandidat pada kedua sisi.

2. Pemicu berbasis *event* dengan isi dari perbandingan

Pendekatan ini memantau perubahan melalui antarmuka notifikasi *filesystem*, lalu menggunakan notifikasi tersebut untuk menentukan berkas mana yang perlu diperiksa. Kelebihan pendekatan ini adalah cakupan pemindaian menyempit dari seluruh pohon direktori menjadi berkas yang tersentuh saja. Akan tetapi, notifikasi hanya menunjukkan berkas mana yang berubah dan

bukan bagian mana dari berkas tersebut, sehingga isi perubahan tetap diperoleh melalui pembacaan ulang dan perbandingan. Pemindaian penuh juga tetap dipertahankan sebagai jaring pengaman ketika notifikasi hilang, sehingga perbandingan *state* tidak benar-benar dihapus.

3. Penangkapan operasi beserta isinya

Pendekatan ini menangkap operasi yang mengubah direktori pada saat operasi tersebut berlangsung, beserta isi *byte* yang dituliskannya, kemudian menyusunnya menjadi artefak yang dapat dipindahkan dan diterapkan ulang oleh program terpisah pada direktori replika. Kelebihan pendekatan ini adalah informasi perubahan sudah tersedia pada saat operasi terjadi sehingga tidak memerlukan pembacaan ulang berkas sumber. Biaya pendekatan ini ditentukan oleh volume perubahan yang benar-benar terjadi. Akan tetapi, ada potensi biaya tambahan yang pada jalur operasi *filesystem* aplikasi. Dari ketiga alternatif solusi tersebut, solusi ini dipilih karena paling sesuai dengan permasalahan yang telah dianalisis dalam penelitian ini.

III.1.3.2 Pemilihan Lapisan Penangkapan Perubahan

Operasi *filesystem* melewati beberapa lapisan sebelum mencapai penyimpanan, dan penangkapan dapat dilakukan pada salah satu di antaranya. Tabel III.3 menunjukkan perbandingan lapisan yang dipertimbangkan untuk digunakan dalam eksperimen ini.

Tabel III.3: Perbandingan lapisan penangkapan perubahan

Lapisan	Kelebihan	Kekurangan
<i>Userspace</i>	+Tidak memerlukan hak istimewa +Tidak menuntut perubahan pada kernel	-Isi perubahan tidak tersedia kecuali program ditempatkan pada jalur I/O aplikasi -Memerlukan modifikasi pada aplikasi yang memodifikasi direktori, tetapi jika hal tersebut dapat diatasi, terdapat potensi overhead akibat <i>mode switching</i> yang diperlukan untuk membaca byte data dari kernel

Tabel III.3: Perbandingan lapisan penangkapan perubahan (lanjutan)

Lapisan	Kelebihan	Kekurangan
<i>System call</i>	+Antarmuka relatif stabil antarversi kernel +Seluruh permintaan aplikasi kepada kernel melewati lapisan ini	-Satu efek yang sama dapat dihasilkan oleh banyak panggilan berbeda sehingga setiap varian harus ditangani terpisah -Yang tersedia berupa deskriptor berkas dan penunjuk memori proses, bukan objek <i>filesystem</i>
VFS	+Berlaku lintas jenis <i>filesystem</i> +Operasi sudah dinyatakan dalam objek <i>filesystem</i> , sehingga isi dan sasaran perubahan tersedia bersamaan	-Titik penautan berupa fungsi internal kernel yang dapat berubah antarversi -Memerlukan hak istimewa
<i>Filesystem</i>	+Isi perubahan tersedia pada tempat penulisan sebenarnya +Seluruh operasi pada <i>filesystem</i> tersebut dijamin melewatinya	-Hanya berlaku untuk satu jenis <i>filesystem</i> -Menuntut perubahan pada implementasi <i>filesystem</i> yang digunakan
Blok	+Transparan terhadap jenis <i>filesystem</i> +Seluruh penulisan ke perangkat penyimpanan melewatinya	-Tidak mengenal berkas maupun direktori -Tidak dapat dilingkupi ke satu direktori tertentu

Dari lapisan-lapisan tersebut, dipilih VFS sebagai lapisan penangkapan untuk eksperimen ini. Alasan utama pemilihannya adalah posisi VFS pada jalur operasi *filesystem*. Di atasnya, operasi masih berupa permintaan aplikasi yang belum dinyatakan dalam objek *filesystem*. Di bawahnya, penangkapan terikat pada satu jenis *filesystem*. Pada lapisan blok, berkas serta direktori bahkan sudah tidak dikenali lagi, sebagaimana terlihat pada DRBD yang dibahas pada bagian II.2.3. VFS menjadi lapisan yang sekaligus berlaku lintas *filesystem* dan masih mengenali berkas serta direktori, sehingga kebutuhan F-1, F-2, serta NF-2 dapat dipenuhi bersamaan.

III.1.3.3 Pemilihan Mekanisme Instrumentasi

Penangkapan pada lapisan VFS menuntut kode pengamat dijalankan di dalam kernel. Tabel III.4 menunjukkan perbandingan mekanisme yang dipertimbangkan untuk digunakan dalam eksperimen ini.

Tabel III.4: Perbandingan mekanisme instrumentasi kernel

Mekanisme	Kelebihan	Kekurangan
Perubahan kode sumber kernel	+Akses penuh tanpa pembatasan +Dapat ditempatkan pada titik mana pun	-Perubahan perlu diajukan dan diterima oleh komunitas pengembang kernel -Menuntut penggunaan kernel hasil kompilasi sendiri
Modul kernel	+Dapat dimuat tanpa mengubah kode sumber kernel +Akses penuh ke struktur data kernel	-Kesalahan pada kode dapat menyebabkan kernel mengalami <i>crash</i> -Perlu dikompilasi ulang untuk setiap versi kernel
eBPF	+Program diperiksa <i>verifier</i> sebelum dimuat sehingga kesalahan tidak menjatuhkan kernel +Tidak menuntut modul tambahan maupun <i>reboot</i>	-Memerlukan hak istimewa -Terbatas pada operasi yang diizinkan <i>verifier</i>

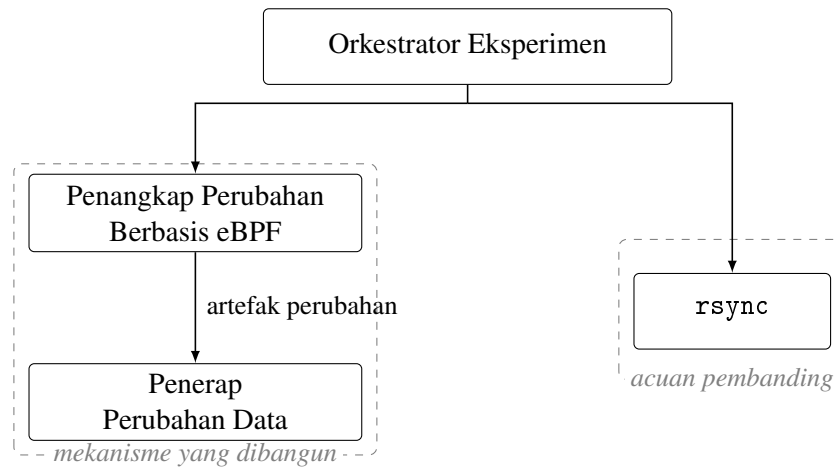
Dari mekanisme-mekanisme tersebut, dipilih eBPF sebagai mekanisme instrumentasi untuk eksperimen ini. Alasan utama pemilihannya adalah eBPF memberikan akses ke lapisan VFS tanpa mengubah kode sumber kernel maupun memuat modul tambahan, sementara keamanannya dijamin oleh *verifier* sebagaimana dijelaskan pada bagian II.3.2. Sebagai konsekuensi, pendekatan ini memerlukan hak istimewa dan titik penautannya terikat pada fungsi internal kernel yang bukan merupakan antarmuka stabil.

III.2 Rancangan

Bagian ini menjelaskan rancangan sistem yang akan diimplementasikan. Rancangan tersebut merujuk pada kebutuhan sistem yang dirumuskan pada bagian III.1.2 serta alternatif solusi yang dipilih pada bagian III.1.3. Pembahasan mencakup struktur sistem, arsitektur sistem, serta alur penentuan perubahan. Komponen dalam bagian ini didefinisikan sebagai satuan fungsional yang berdiri sendiri serta dapat dipasangkan dengan komponen lain.

III.2.1 Rancangan Struktural

Sistem eksperimen dirancang sebagai empat komponen. Tiga komponen pertama menjalankan replikasi direktori, sedangkan komponen keempat mengukur ketiganya. Gambar III.1 menunjukkan susunan keempat komponen tersebut.



Gambar III.1: Susunan komponen sistem eksperimen

Komponen pertama adalah penangkap perubahan berbasis eBPF. Komponen ini merekam operasi yang mengubah isi maupun struktur direktori sumber, tepat ketika operasi tersebut berlangsung. Rekaman yang terkumpul disusun menjadi artefak perubahan. Tiga keputusan rancangan melekat pada komponen ini. Keputusan pertama adalah membatasi perekaman pada satu subpohon direktori, sehingga operasi di luar direktori target tidak ikut terekam. Keputusan kedua adalah menyediakan pengambilan artefak secara inkremental, sehingga setiap pengambilan hanya memuat perubahan sejak pengambilan sebelumnya. Keputusan ketiga adalah menolak menerbitkan artefak ketika perekaman diketahui tidak lengkap. Keputusan ketiga tersebut lahir dari hilangnya mekanisme koreksi yang melekat pada pemindaian, sebagaimana dibahas pada bagian III.1.3.1.

Komponen kedua adalah penerap perubahan data. Komponen ini membaca artefak lalu menerapkan setiap rekaman di dalamnya pada direktori replika. Komponen ini dirancang agar tidak pernah mengakses direktori sumber. Artefak karena itu harus bersifat mandiri, yaitu memuat seluruh keterangan yang dibutuhkan untuk memulihkan kondisi direktori di mesin utama ke mesin replika.

Komponen ketiga adalah *rsync*. Komponen ini menentukan perubahan melalui pemindaian serta perbandingan kondisi dua direktori sebagaimana dianalisis pada bagian III.1.3.1. Komponen ini menghasilkan pengaruh yang sama dengan kedua komponen sebelumnya, yaitu direktori replika menjadi setara dengan direktori sumber. Perbedaannya terletak pada cara perubahan tersebut ditentukan.

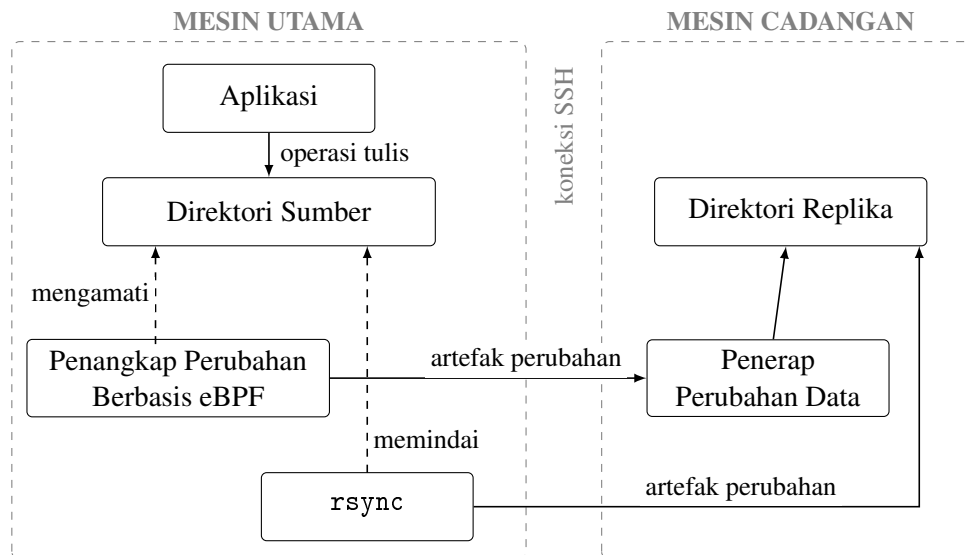
Komponen keempat adalah orkestrator eksperimen. Komponen ini menjalankan seluruh percobaan secara otomatis. Orkestrator menyiapkan keadaan awal, menja-

lankan beban tulis, memicu sinkronisasi, mengukur waktunya, memverifikasi kesetaraan kedua direktori, lalu mencatat seluruh angkanya. Dua keputusan rancangan melekat pada komponen ini. Keputusan pertama adalah memicu sinkronisasi melalui penjadwalan yang dikendalikan orkestrator sendiri. Interval sinkronisasi merupakan variabel yang hendak diamati. Keputusan kedua adalah menempatkan kedua mekanisme yang dibandingkan di balik antarmuka yang sama. Antarmuka tersebut terdiri atas empat operasi, yaitu memulai perekaman, membuang perubahan yang sudah terkumpul, menjalankan satu sinkronisasi yang diukur, serta menghentikan perekaman. Orkestrator memanggil keempat operasi itu dengan urutan yang sama untuk kedua mekanisme.

Merujuk kebutuhan pada Tabel III.1 serta Tabel III.2, penangkap perubahan berbasis eBPF memenuhi kebutuhan F-1, F-2, F-3, serta F-4. Penerap perubahan data memenuhi kebutuhan F-5. `rsync` memenuhi kebutuhan F-6. Orkestrator eksperimen memenuhi kebutuhan F-7, F-8, F-9, serta F-10. Kebutuhan NF-1 dipenuhi bersama oleh penangkap beserta penerap, kebutuhan NF-2 dan NF-3 dipenuhi oleh penangkap, sedangkan kebutuhan NF-4 dan NF-5 dipenuhi oleh orkestrator.

III.2.2 Arsitektur Sistem

Keempat komponen tersebut tersebar pada dua mesin yang terpisah. Pemisahan tersebut mengikuti premis penelitian ini, yaitu replika harus tetap tersedia ketika mesin yang menyimpan data asli mengalami kegagalan. Gambar III.2 menunjukkan penempatan setiap komponen beserta lintasan artefak di antara kedua mesin.



Gambar III.2: Arsitektur sistem eksperimen pada dua mesin

Mesin utama memuat direktori sumber beserta aplikasi yang menulis ke dalamnya. Penangkap perubahan berbasis eBPF ikut ditempatkan pada mesin tersebut karena isi tulisan hanya tersedia pada saat operasi berlangsung. `rsync` juga ditempatkan pada mesin yang sama karena pemindaian menuntut pembacaan langsung atas direktori sumber. Mesin cadangan memuat direktori replika beserta penerap perubahan data.

Kedua mekanisme menyeberangkan perubahan data melalui koneksi yang sama. Aplikasi pada gambar tersebut tidak dimodifikasi sama sekali. Direktori sumber juga tetap berupa direktori biasa pada *filesystem* yang sudah ada. Penangkap perubahan berbasis eBPF mengamati operasi tulis dari dalam kernel, sehingga aplikasi tidak perlu mengetahui keberadaannya. Sifat tersebut merupakan realisasi dari kebutuhan NF-2.

III.2.3 Alur Penentuan Perubahan

Kedua mekanisme menghasilkan pengaruh akhir yang sama, yaitu direktori replika menjadi setara dengan direktori sumber. Keduanya menempuh alur yang berbeda untuk mencapainya. Gambar III.3 menyandingkan kedua alur tersebut.



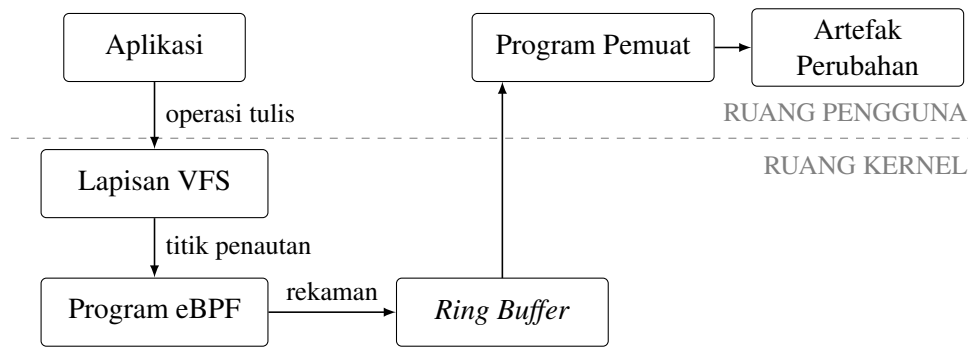
Gambar III.3: Perbandingan alur penentuan perubahan pada kedua mekanisme

Alur penangkapan bermula dari operasi tulis aplikasi. Program eBPF merekam operasi tersebut beserta isi tulisannya pada saat operasi berlangsung. Program pemuat menyusun rekaman yang terkumpul menjadi artefak. Orkestrator mengambil artefak tersebut ketika sinkronisasi dipicu, lalu mengirimkannya ke mesin cadangan. Penerap menerapkan artefak itu pada direktori replika. Isi perubahan sudah dipegang sejak langkah kedua, sehingga alur ini tidak pernah membaca ulang direktori sumber.

Alur pemindaian bermula dari pemicu sinkronisasi. rsync menelusuri pohon direktori pada kedua mesin, membaca berkas kandidat, lalu menentukan bagian yang berbeda di antara keduanya. Bagian tersebut kemudian ditulis pada direktori replika. Ketiga langkah tengah pada alur ini menyentuh seluruh pohon direktori, bukan hanya bagian yang berubah.

III.2.4 Rancangan Penangkap Perubahan Berbasis eBPF

Penangkap perubahan berbasis eBPF merupakan komponen utama yang dibangun pada penelitian ini. Program tersebut terdiri atas dua program yang hidup di dua tempat yang berbeda. Salah satu program berjalan di *userspace*, sedangkan yang lainnya berjalan di *kernel space*. Gambar III.4 menunjukkan struktur komponen tersebut beserta aliran data di antara kedua programnya.



Gambar III.4: Struktur penangkap perubahan berbasis eBPF

Pembagian tugas di antara kedua program mengikuti batas tersebut karena dua alasan. Alasan pertama, isi tulisan hanya tersedia di dalam kernel pada saat operasi berlangsung, sehingga pengambilannya harus dikerjakan di sana. Alasan kedua, penyusunan artefak menuntut alokasi memori yang besarnya tidak diketahui di muka serta penulisan berkas, sedangkan *verifier* eBPF tidak mengizinkan keduanya.

Program eBPF ditautkan pada sejumlah fungsi di lapisan VFS. Setiap kali salah satu fungsi tersebut dipanggil aplikasi, program eBPF membaca kunci *inode* dari direktori induk objek yang disentuh, lalu mencari kunci itu pada *hash map* lingkup. Program eBPF menerbitkan rekaman apabila kuncinya ditemukan. Program eBPF mengabaikan operasi tersebut apabila kuncinya tidak ditemukan. Rekaman yang diterbitkan memuat kunci *inode* induk, nama entri yang disentuh, serta isi *byte* yang dituliskan.

Rekaman tersebut menyeberang ke ruang pengguna melalui *ring buffer*, yaitu antrean melingkar yang hanya mengalirkan data satu arah. Antrean tersebut dipilih karena penulisan ke dalamnya tidak menunggu pembacanya, sehingga operasi aplikasi yang sedang berlangsung tidak tertahan oleh kecepatan program pemuat dalam mengosongkannya.

Program pemuat mengerjakan tiga hal. Tugas pertama adalah memuat program eBPF ke dalam kernel ketika penangkap dijalankan. Program kemudian membaca *ring buffer* secara terus-menerus. Setiap rekaman yang diterima belum memuat *path* sasaran, sehingga program pemuat menyusunnya menggunakan tabel *path* yang dipeliharanya sendiri. Rekaman yang sudah lengkap dikumpulkan menjadi artefak perubahan.

BAB IV

IMPLEMENTASI DAN EVALUASI

Bab ini memaparkan implementasi dari rancangan yang telah disusun pada Bab III beserta hasil evaluasinya. Pembahasan dimulai dari lingkungan tempat sistem dibangun serta diukur, dilanjutkan dengan rincian implementasi setiap komponen, lalu ditutup dengan evaluasi yang mencakup pengujian fungsional serta analisis atas data eksperimen.

Sepanjang bab ini istilah **artefak perubahan** merujuk pada berkas yang menyimpan rangkaian perubahan dari direktori sumber. Berkas tersebut dapat dikirim ke mesin lain lalu diterapkan pada direktori replika tanpa perlu membaca kembali direktori sumbernya.

IV.1 Lingkungan

Premis penelitian ini adalah tersedianya salinan data ketika mesin yang menyimpan data asli mengalami kegagalan. Replika yang ditempatkan pada mesin yang sama dengan sumbernya tidak memenuhi premis tersebut karena keduanya ikut hilang ketika mesin tersebut gagal. Oleh karena itu, direktori sumber serta direktori replika ditempatkan pada dua mesin fisik yang terpisah sehingga setiap artefak perubahan harus benar-benar menyeberangi jaringan sebelum dapat diterapkan.

Kedua mesin memiliki peran yang berbeda. Mesin pertama berperan sebagai mesin utama yang memuat direktori sumber, sedangkan mesin kedua berperan sebagai mesin cadangan. Seluruh eksperimen dikendalikan dari mesin utama. Kedua mesin berada pada perangkat keras yang identik sehingga perbedaan hasil tidak berasal dari perbedaan spesifikasi. Spesifikasi lingkungan yang digunakan dapat dilihat pada Tabel IV.1.

Tabel IV.1: Lingkungan implementasi dan pengukuran

Komponen	Spesifikasi
Jumlah mesin	2
Prosesor	AMD EPYC 9354P, 32 <i>core</i> , 3,25 GHz

Tabel IV.1: Lingkungan implementasi dan pengukuran (lanjutan)

Komponen	Spesifikasi
Memori	192 GB DDR5-4800 ECC
Penyimpanan	2 unit NVMe SSD 800 GB antarmuka PCIe 4.0
Antarmuka jaringan eksperimen	Mellanox ConnectX-6 100 Gb, PCIe 4.0
Sistem operasi	Linux dengan kernel 6.8.0-117-generic
Acuan pembandingan	rsync versi 3.2.7 protokol 31
Dependensi	clang/LLVM, libelf, zlib
Bahasa orkestrator eksperimen	Python 3
Aplikasi nyata yang diuji	SQLite, PostgreSQL 16, MySQL 8

Kernel yang digunakan memenuhi dua syarat yang dituntut mekanisme penangkapan sebagaimana dibahas pada bagian III.1.3.3. Syarat pertama adalah tersedianya BTF, yaitu keterangan mengenai tata letak struktur data di dalam kernel. Program eBPF memakai keterangan tersebut untuk membaca isi struktur kernel tanpa perlu mengetahui tata letaknya sendiri sehingga program yang sama tetap benar meskipun tata letak itu berubah antarversi kernel. Syarat kedua adalah dukungan *fentry* dan *fexit*, yaitu cara menautkan program eBPF pada awal serta akhir sebuah fungsi kernel dengan biaya yang jauh lebih rendah daripada *kprobe*.

IV.2 Implementasi

Bagian ini menjelaskan implementasi dari empat komponen yang dibangun, yaitu program penangkap perubahan berbasis eBPF, penerap perubahan data, rsync sebagai acuan pembandingan, serta orkestrator eksperimen yang mengukur kedua mekanisme tersebut. Penjelasan diawali dengan batasan implementasi beserta kakas yang digunakan.

IV.2.1 Batasan Implementasi

Berikut adalah batasan implementasi yang ditetapkan dalam pengembangan sistem.

1. Objek yang direplikasi hanya berkas reguler beserta direktori. Entri *filesystem* khusus seperti *symbolic link*, *hard link*, *device file*, serta *Unix domain socket* tidak termasuk dalam cakupan implementasi.
2. Kesetaraan antara direktori sumber dengan direktori replika didefinisikan berdasarkan kesamaan struktur direktori serta kesamaan isi *byte* berkas. Meta-

data seperti izin akses, kepemilikan, *timestamp*, atribut tambahan, serta ACL tidak menjadi kriteria kesetaraan.

3. Replikasi bersifat satu arah dari direktori sumber menuju direktori replika. Kesetaraan dijamin setelah seluruh artefak perubahan selesai diterapkan.
4. Penangkapan perubahan memerlukan hak akses *root*, kernel yang menyediakan informasi tipe BTF, serta dukungan penautan pada fungsi internal kernel.

IV.2.2 Kakas yang Digunakan

Kakas yang digunakan dalam implementasi ini adalah sebagai berikut.

1. **Bahasa C beserta pustaka libbpf.** Digunakan untuk menulis program eBPF yang berjalan di dalam kernel serta program pemuatnya di *userspace*. Perkakas *bpftool* digunakan untuk menghasilkan berkas *skeleton* yang menghubungkan keduanya.
2. **Bahasa Python 3.** Digunakan untuk membangun orkestrator eksperimen yang mengelola beban kerja, memicu sinkronisasi, mencatat metrik, serta memverifikasi kesetaraan direktori.
3. *rsync.* Digunakan sebagai acuan pembandingan yang menentukan perubahan melalui pemindaian serta perbandingan *state*.
4. **Bahasa pemrograman Go.** Digunakan untuk membangun aplikasi *todo list* sederhana sebagai representasi perangkat lunak nyata yang mampu menerima permintaan HTTP dan melakukan penulisan ke basis data SQLite, PostgreSQL, serta MySQL.
5. **SQLite, PostgreSQL 16, serta MySQL 8.** Digunakan sebagai basis data untuk menguji kemampuan mekanisme yang dibangun dalam menangkap perubahan yang dilakukan oleh perangkat lunak secara generik.
6. **matplotlib.** Digunakan untuk mengolah berkas hasil pengukuran menjadi gambar yang disajikan pada bagian IV.3.3.

IV.2.3 Implementasi Penangkap Perubahan Berbasis eBPF

Penangkap perubahan berbasis eBPF terdiri atas dua program yang bekerja bersama. Program pertama ditulis dalam eBPF dan berjalan di dalam kernel. Program ini mengamati operasi *filesystem* tepat ketika operasi tersebut berlangsung. Program

kedua berjalan di *userspace* dan selanjutnya disebut program pemuat. Program pemuat memuat program eBPF ke dalam kernel, menerima rekaman yang dikirim program eBPF, lalu menyusun rekaman tersebut menjadi artefak perubahan.

IV.2.3.1 Penentuan Objek yang Direkam

Kebutuhan F-2 pada Tabel III.1 menuntut penangkap hanya merekam operasi yang terjadi di dalam direktori target. Subdirektori yang dibuat ketika sistem sedang berjalan juga harus ikut terekam. Cara paling sederhana untuk memenuhi kebutuhan itu adalah menyusun *path* lengkap dari setiap objek yang disentuh, lalu mencocokkan *path* tersebut dengan *path* direktori target. Implementasi ini tidak memakai cara tersebut karena empat alasan.

Alasan pertama, kernel tidak menyimpan *path* lengkap di mana pun. Kernel mewakili setiap entri direktori dengan sebuah struktur bernama *dentry*. Satu *dentry* hanya mengetahui namanya sendiri beserta penunjuk menuju *dentry* induknya. Program yang ingin memperoleh *path* lengkap harus menaiki rantai penunjuk itu satu tingkat demi satu tingkat sampai akar *mount*, lalu merangkai nama yang terkumpul dalam urutan terbalik.

Alasan kedua, *verifier* eBPF melarang perulangan semacam itu. Jumlah iterasinya bergantung pada kedalaman direktori, sedangkan *verifier* menuntut setiap program selesai dijalankan sebagaimana dijelaskan pada bagian II.3.2, sehingga batas iterasinya harus sudah diketahui pada saat kompilasi. Program eBPF karena itu harus mematok kedalaman direktori pada suatu nilai tetap. Pohon direktori yang lebih dalam daripada patokan tersebut gagal diselesaikan. Setiap tingkat pendakian juga membaca memori kernel yang pembacaannya dapat gagal, sedangkan hasilnya harus dirangkai secara mundur ke dalam *buffer* berukuran tetap.

Alasan ketiga, fungsi pembantu yang disediakan kernel tidak dapat dipakai pada semua titik penautan. Kernel memang menyediakan fungsi untuk menyusun *path* dari dalam program eBPF. Kernel membatasi pemakaian fungsi tersebut pada sekumpulan titik penautan tertentu yang tidak mencakup jalur tulis. Mekanisme ini justru menautkan diri pada jalur tulis, sehingga fungsi itu tidak dapat dipakai secara seragam.

Alasan keempat, satu *path* tidak selalu menunjuk satu objek. Sebuah berkas dapat

dijangkau melalui lebih dari satu *path* ketika terdapat *bind mount*. Pencocokan *path* karena itu berpotensi melewati objek yang sebenarnya berada di dalam direktori target.

Implementasi ini karena itu mengenali setiap objek melalui pasangan dev serta ino. Pasangan tersebut selanjutnya disebut kunci *inode*.

Program eBPF membaca dev dari *superblock*, yaitu struktur kernel yang mewakili satu *filesystem* yang sedang terpasang. Nilai dev karena itu menunjuk *filesystem* tempat berkas berada, bukan *block device* tempat *filesystem* tersebut tersimpan. Satu *block device* yang dipartisi menjadi beberapa *filesystem* menghasilkan dev yang berbeda-beda. Sebaliknya, *filesystem* semu seperti *tmpfs* tetap memperoleh dev meskipun tidak bertumpu pada *block device* sama sekali.

Program eBPF membaca ino dari *inode* berkas. Nilai tersebut unik di dalam satu *filesystem*. Program eBPF membaca kedua nilai itu langsung dari struktur kernel dalam waktu tetap, tanpa perulangan sama sekali. Kedua nilai juga tidak berubah ketika berkas berpindah direktori, sepanjang perpindahannya masih di dalam *filesystem* yang sama.

Penangkap menyimpan kunci *inode* dari setiap direktori yang diawasi ke dalam sebuah *hash map* di dalam kernel. *Hash map* adalah struktur data kunci-nilai yang disediakan kernel bagi program eBPF.

Program eBPF memakai *hash map* tersebut untuk memutuskan apakah sebuah operasi perlu direkam. Setiap kali operasi terjadi, program eBPF membaca kunci *inode* dari direktori induk objek yang disentuh, lalu mencari kunci itu di dalam *hash map*. Program eBPF merekam operasi tersebut apabila kuncinya ditemukan. Program eBPF mengabaikannya apabila kuncinya tidak ditemukan.

Program pemuat mengisi *hash map* tersebut untuk pertama kali ketika penangkap dijalankan. Program pemuat menelusuri direktori target satu kali, lalu memasukkan kunci *inode* dari setiap direktori yang ditemukannya. Setelah itu program eBPF sendiri yang memperbarui isinya. Program eBPF menambahkan kunci ketika aplikasi membuat direktori baru maupun memindahkan direktori ke dalam target. Program eBPF membuang kunci ketika aplikasi menghapus direktori maupun memindahkan direktori ke luar target. Subdirektori yang lahir ketika sistem sedang berjalan dengan demikian ikut terawasi tanpa pencocokan *path* sama sekali.

Artefak perubahan tetap harus menyebutkan *path* sasaran dari setiap operasi. Program pemuat yang mengerjakan penyusunan *path* tersebut, bukan program eBPF. Setiap rekaman yang dikirim program eBPF hanya membawa dua hal, yaitu kunci *inode* direktori induk beserta nama entri yang disentuh. Program pemuat menyimpan tabel yang memetakan kunci *inode* setiap direktori ke *path* relatifnya. Program pemuat mencari kunci induk pada tabel tersebut, lalu menyambungkannya dengan nama entri sehingga *path* lengkap dari objek yang berubah kembali terbentuk.

Sebagai ilustrasi, misalkan direktori target adalah `/var/tmp/exp/src`. Direktori tersebut memuat satu subdirektori bernama `data` yang berisi satu berkas bernama `tasks.db`. Gambar IV.1 menunjukkan apa yang dikerjakan kedua program ketika aplikasi menulis ke berkas tersebut.

```
Hash map lingkup di dalam kernel
(271581192, 29369023)
(271581192, 29369024)

Tabel path di userspace
(271581192, 29369023) -> .
(271581192, 29369024) -> data

Aplikasi menulis ke /var/tmp/exp/src/data/tasks.db
kunci inode induk      (271581192, 29369024)
periksa hash map lingkup ditemukan
rekaman ke userspace   (271581192, 29369024) + "tasks.db"
cari pada tabel path   "data"
path pada artefak      data/tasks.db

Aplikasi menulis ke /var/log/app.log
kunci inode induk      (271581192, 20986191)
periksa hash map lingkup tidak ditemukan
rekaman                tidak diterbitkan
```

Gambar IV.1: Ilustrasi pembatasan lingkup perekaman beserta penyusunan *path*

Dua hal terlihat dari ilustrasi tersebut. Pertama, *hash map* di dalam kernel sama sekali tidak menyimpan *path*. *Hash map* tersebut hanya menyimpan kunci, sehingga program eBPF memang tidak pernah perlu menyusun *path*. Kedua, `/var/tmp/exp/src/data` beserta `/var/log` berada pada *filesystem* yang sama sehingga dev keduanya pun sama. Meskipun begitu, hanya operasi pada direktori pertama yang direkam. Penangkap dengan demikian membatasi rekamannya pada tingkat direktori, bukan pada tingkat *filesystem*.

Rancangan ini memenuhi kebutuhan F-2 karena penangkap hanya merekam ope-

rasi di dalam direktori yang telah ditentukan. Rancangan ini sekaligus memenuhi kebutuhan NF-3 karena penangkap tidak membaca ulang isi direktori sumber untuk menentukan perubahan.

IV.2.3.2 Titik Penautan pada Lapisan VFS

Kebutuhan F-1 menuntut sistem untuk merekam operasi mutasi beserta isi perubahan yang dihasilkannya. Isi tulisan hanya tersedia pada saat operasi berlangsung, sehingga titik penautan harus dipilih sedemikian rupa sehingga seluruh jalur yang dapat mengubah isi maupun struktur direktori benar-benar terlewati. Tabel IV.2 merangkum titik penautan yang dipakai beserta alasan pemilihannya.

Tabel IV.2: Titik penautan program penangkap pada lapisan VFS

Operasi	Titik penautan	Alasan pemilihan
Pembuatan berkas	<code>do_filp_open</code> pada sisi keluar	Panggilan <code>open</code> dengan penanda pembuatan berkas pada <code>ext4</code> memanggil operasi pembuatan milik <i>inode</i> secara langsung sehingga melewati <code>vfs_create</code>
Penulisan biasa	<code>vfs_write</code> pada sisi masuk serta sisi keluar	Jalur tulis yang paling umum. SQLite menuliskan berkas basis data beserta <i>write-ahead log</i> miliknya melalui jalur ini
Penulisan vektor	<code>vfs_writew</code> pada sisi masuk serta sisi keluar	Panggilan <code>writew</code> beserta <code>pwritew</code> mencapai operasi tulis <i>inode</i> melalui fungsi yang disisipkan kompilator sehingga tidak terlihat oleh penautan berbasis BTF. PostgreSQL melepas beberapa halaman <i>write-ahead log</i> sekaligus melalui jalur ini
Penulisan asinkron <i>native</i>	<code>io_submit_one</code> yang dipasangkan dengan <code>aio_complete_rw</code>	Penulisan melalui pustaka AIO <i>native</i> tidak melewati <code>vfs_write</code> maupun <code>vfs_writew</code> sama sekali. InnoDB menuliskan halaman berkas data melalui jalur ini
Pemotongan berkas	<code>do_truncate</code> pada sisi masuk serta sisi keluar	Satu-satunya fungsi yang pasti dilewati oleh <code>truncate</code> , <code>ftruncate</code> , serta <code>open</code> dengan penanda pemotongan
Alokasi ruang berkas	<code>vfs_fallocate</code> pada sisi masuk serta sisi keluar	Dapat mengubah ukuran berkas maupun menolak rentang <i>byte</i> tanpa menulis satu <i>byte</i> pun sekaligus tanpa melewati <code>do_truncate</code> . InnoDB memperbesar <i>tablespace</i> melalui jalur ini
Perubahan struktur direktori	<code>vfs_mkdir</code> , <code>vfs_rmdir</code> , <code>vfs_unlink</code> , serta <code>vfs_rename</code> pada sisi masuk serta sisi keluar	Seluruh perubahan struktur direktori melewati fungsi-fungsi tersebut. Penautan pada <code>vfs_rename</code> bersifat menanggung beban karena PostgreSQL mendaur ulang segmen <i>write-ahead log</i> dengan menamai ulang berkas sementara ke tempatnya

Program eBPF menautkan diri pada dua titik untuk setiap operasi pada Tabel IV.2, yaitu ketika fungsi kernel mulai dijalankan serta ketika fungsi tersebut selesai. Dua titik diperlukan karena informasi yang dibutuhkan tersebar pada keduanya. Berkas yang dituju operasi, posisi penulisan, banyaknya *byte* yang akan ditulis, serta alamat memori tempat isi tulisan berada hanya tersedia pada titik masuk. Nilai yang menunjukkan berhasil atau gagal nya operasi hanya tersedia pada titik keluar.

Banyak permintaan dapat berjalan bersamaan di dalam kernel, sehingga program eBPF tidak dapat menyimpan argumen dari titik masuk pada satu tempat bersama. Program eBPF menyimpan argumen tersebut pada sebuah *hash map* sementara. Kuncinya adalah penanda unik pemanggil, yaitu gabungan nomor proses beserta nomor *thread* yang sedang menjalankan operasi itu. Titik keluar memakai penanda yang sama untuk mengambil kembali argumen milik panggilan yang sesuai.

Program eBPF baru menyusun rekaman perubahan data pada titik keluar, itu pun hanya apabila nilai baliknya menunjukkan operasi berhasil. Operasi yang gagal, misalnya penulisan yang ditolak karena ruang penyimpanan habis, dengan demikian tidak pernah masuk ke dalam artefak perubahan.

Program eBPF sengaja tidak menautkan diri pada *fsync*, *fdatasync*, serta *sync_file_range*. Ketiganya tidak mengubah isi maupun struktur direktori, melainkan hanya memerintahkan kernel agar isi yang sudah tertulis dipindahkan dari *cache* halaman menuju media penyimpanan. Isi tersebut sudah tersalin ke dalam artefak perubahan pada saat operasi tulisnya dieksekusi, yaitu sebelum sinkronisasi dilakukan, sehingga menautkan operasi sinkronisasi tidak akan menambahkan satu *byte* pun pada artefak. Program penerap juga tidak memerlukan padanan ketiganya karena yang dijamin penelitian ini adalah kesetaraan isi berkas, bukan jaminan durabilitas pada replika.

IV.2.3.3 Pengambilan Artefak dan Perilaku ketika Perekaman Tidak Lengkap

Program eBPF menyusun satu rekaman perubahan data setiap kali ia memutuskan sebuah operasi perlu direkam. Rekaman tersebut memuat jenis operasinya, sasaran yang disentuh, serta isi *byte* yang dituliskan apabila operasinya berupa penulisan.

Program eBPF mengirimkan setiap rekaman ke *userspace* melalui *ring buffer*, yaitu antrean melingkar yang dibaca terus-menerus oleh program pemuat yang hidun di *userspace*. Program tersebut memiliki tugas untuk mengumpulkan rekaman yang masuk menjadi satu *batch* di dalam memori, dengan urutan yang sama seperti urutan terjadinya operasi.

Orkestrator eksperimen mengambil *batch* tersebut melalui *socket* domain Unix. Satu permintaan dilakukan untuk mendapatkan seluruh isi *batch* sekaligus dan mengosongkannya sehingga pengambilan berikutnya hanya berisi perubahan yang terjadi setelah pengambilan sebelumnya. Rancangan ini memenuhi kebutuhan F-3 karena artefak yang dihasilkan bersifat mandiri serta dapat dipindahkan ke mesin lain. Rancangan ini sekaligus memenuhi kebutuhan F-4 karena setiap pengambilan hanya menyerahkan perubahan yang baru.

Menghapus pemindaian berarti menghapus pula mekanisme koreksi yang melekat padanya. *rsync* tidak mungkin melewatkan perubahan secara permanen karena yang diamatinya adalah keadaan akhir direktori. Perubahan yang belum tertangani pada satu siklus tetap terlihat pada siklus berikutnya. Penangkap tidak memiliki jaring pengaman semacam itu karena yang diamatinya adalah operasi yang sedang berlangsung. Satu operasi yang terlewat tidak akan muncul kembali, sehingga replika menjadi salah secara permanen. Hal tersebut adalah kekurangan yang dimiliki oleh eBPF untuk saat ini.

Program eBPF memelihara sekumpulan penghitung yang mencatat setiap peristiwa kehilangan data, antara lain kegagalan menyediakan ruang pada *ring buffer*, kegagalan membaca memori, isi tulisan yang hanya tersalin sebagian, kegagalan pada jalur penulisan asinkron, serta kemunculan mode *fallocate* yang tidak didukung. Program pemuat memeriksa seluruh penghitung tersebut pada setiap pengambilan *batch*. Apabila satu saja bernilai bukan nol, program pemuat menolak menyerahkan artefak lalu memberikan pesan error pada program yang meminta artefak.

IV.2.4 Implementasi Program Penerap Perubahan Data

Program penerap perubahan membaca artefak lalu menerapkan setiap rekaman di dalamnya pada sebuah direktori target. Artefak diawali oleh *header* yang memuat penanda format, versi, jumlah rekaman, serta ukuran muatan. Setelah itu menyu-

sul rekaman-rekaman yang tersusun menurut urutan kronologis terjadinya operasi. Setiap rekaman memuat jenis operasi, *path* sasaran, *path* baru apabila operasinya berupa penamaan ulang, serta isi *byte* apabila operasinya berupa penulisan.

Program ini tidak memuat program eBPF sama sekali serta tidak memerlukan hak istimewa, karena seluruh pekerjaannya dilakukan melalui *system call* biasa. Sifat tersebut penting karena mesin cadangan dengan demikian tidak perlu menyediakan kernel dengan dukungan khusus maupun hak akses *root*.

Setiap *path* yang berasal dari artefak diperiksa terlebih dahulu sehingga *path* absolut serta *path* yang menelusuri keluar menuju direktori induk ditolak. *Path* yang lolos pemeriksaan digabungkan di bawah akar direktori target, sehingga penerapan artefak tidak pernah menulis ke luar pohon direktori tersebut.

Program penerap tidak pernah mengakses direktori sumber sehingga kebutuhan F-5 terpenuhi. Penerapannya juga tidak menuntut jenis *filesystem* tertentu karena seluruh operasinya dinyatakan dalam *system call* yang berlaku umum, sehingga kebutuhan NF-2 terpenuhi pada sisi replika.

IV.2.5 Implementasi rsync sebagai Acuan Pembanding

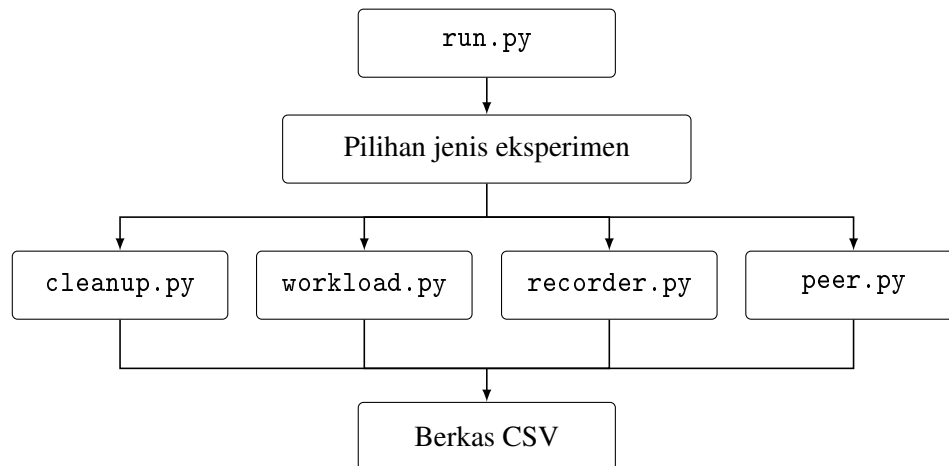
Kebutuhan F-6 menuntut tersedianya acuan pembanding yang menentukan perubahan data menggunakan mekanisme pemindaian serta perbandingan kondisi dua direktori. Mekanisme tersebut diwujudkan menggunakan *rsync* sebagaimana dianalisis pada bagian III.1.3.1.

Dua keputusan konfigurasi ditetapkan agar perbandingan antara kedua mekanisme tetap setara. Keputusan pertama adalah tidak memelihara salinan pembanding di direktori sumber secara lokal. Direktori replika itu sendiri yang menjadi dasar perbandingan, sehingga setiap sinkronisasi benar-benar menelusuri pohon direktori pada kedua sisi lalu membaca berkas kandidat untuk menentukan bagian mana yang berbeda.

Keputusan kedua adalah menyeberangkan artefak kedua mekanisme melalui koneksi SSH yang sama. Orkestrator membuka koneksi tersebut satu kali lalu menggunakannya ulang untuk seluruh titik ukur, sehingga biaya pembukaan koneksi baru tidak ikut terukur pada setiap sinkronisasi. Dengan demikian selisih waktu yang terukur berasal dari cara perubahan ditentukan, bukan dari perbedaan jalur pengangkutan.

IV.2.6 Implementasi Orkestrator Eksperimen

Orkestrator adalah program yang menjalankan seluruh percobaan secara otomatis. Program tersebut menyiapkan keadaan awal, menjalankan beban tulis, memicu sinkronisasi, mengukur waktunya, memverifikasi hasilnya, lalu mencatat seluruh angkanya. Gambar IV.2 menunjukkan strukturnya. Pengguna menentukan jenis percobaan beserta parameternya, lalu orkestrator menjalankan keempat berkas pendukung menurut urutan tetap dan menuliskan hasilnya ke berkas CSV.



Gambar IV.2: Struktur orkestrator eksperimen

Keempat berkas di baris bawah memegang satu tugas masing-masing. Berkas `cleanup.py` menghentikan proses sisa, menghapus direktori kerja, lalu mengosongkan *cache* halaman pada kedua mesin. Berkas `workload.py` membangun dan mengirim beban kerja untuk masing-masing eksperimen. Berkas `peer.py` membuka koneksi SSH ke mesin replika, menyalin direktori sumber ke replika sebelum pengukuran, lalu membandingkan kedua direktori sesudahnya. Berkas `recorder.py` memuat kedua mekanisme yang dibandingkan dalam dua kelas terpisah, yaitu satu untuk `rsync` serta satu untuk penangkap eBPF. Keduanya menyediakan empat operasi yang sama, yaitu memulai perekaman, membuang perubahan yang sudah terkumpul, menjalankan satu sinkronisasi yang diukur, serta menghentikan perekaman. Orkestrator memanggil keempat operasi itu dengan urutan yang sama untuk kedua mekanisme, sehingga kesamaan perlakuan di antara keduanya dijamin oleh struktur programnya. Sifat tersebut merupakan realisasi dari kebutuhan NF-5.

IV.3 Evaluasi

Bagian ini memaparkan hasil evaluasi terhadap sistem yang telah diimplementasikan. Evaluasi dibagi menjadi tiga bagian, yaitu penjelasan mengenai *setup* pengujian, pengujian yang memastikan sistem bekerja sesuai kebutuhan, serta analisis atas data eksperimen.

IV.3.1 Konfigurasi Pengujian

Eksperimen dijalankan di dua mesin berbeda. Salah satu mesin berperan sebagai mesin utama, sedangkan mesin lainnya berperan sebagai mesin replika. Direktori kerja pada kedua mesin ditempatkan pada disk lokal masing-masing mesin sehingga direktori sumber beserta direktori replika benar-benar merupakan dua penyimpanan yang terpisah. Orkestrator eksperimen memeriksa syarat tersebut sebelum eksperimen dimulai lalu menolak berjalan apabila syaratnya tidak terpenuhi.

Eksperimen pertama mengukur biaya sinkronisasi terhadap ukuran *state* yang sudah ada. Volume perubahan dikunci pada 4 KB sedangkan ukuran berkas divariasikan dari 10 MB hingga 1000 MB. Perintah yang dijalankan dapat dilihat pada Gambar IV.3.

```
sudo python3 run.py a2 \  
  --arms ebpf,rsync \  
  --peer panjisri@10.10.1.4 \  
  --work-dir /var/tmp/exp \  
  --sizes 10,50,200,500,1000 \  
  --trials 3 \  
  --fresh
```

Gambar IV.3: Perintah untuk menjalankan eksperimen pertama

Eksperimen kedua mengukur biaya sinkronisasi terhadap banyaknya penulisan ke rentang *byte* yang sama sebelum sinkronisasi dilakukan. Berkas dikunci pada ukuran 10 MB sedangkan banyaknya penulisan divariasikan dari satu kali hingga sepuluh ribu kali. Perintah yang dijalankan dapat dilihat pada Gambar IV.4.

```
sudo python3 run.py a3 \  
  --arms ebpf,rsync \  
  --peer panjisri@10.10.1.4 \  
  --work-dir /var/tmp/exp \  
  --file-size-mb 10 \  
  --repeats 1,10,100,1000,10000 \  
  --trials 3 \  
  --fresh
```

Gambar IV.4: Perintah untuk menjalankan eksperimen kedua

Eksperimen ketiga dan keempat mengukur biaya yang ditimbulkan oleh eBPF akibat berada di dalam jalur eksekusi aplikasi. Keduanya dijalankan pada satu mesin tanpa replika karena yang diukur adalah latensi setiap operasi ketika penangkapan dimatikan dibandingkan ketika penangkapan dihidupkan. Eksperimen ketiga mengukur latensi setiap penulisan data sebesar 4 KB pada berkas biasa, sedangkan eksperimen keempat mengukur latensi setiap permintaan HTTP aplikasi *todo list* dengan SQLite sebagai basis data. Perintah yang dijalankan dapat dilihat pada Gambar IV.5.

```
sudo python3 run.py cdf-fs --work-dir /mnt/exp --ops 5000 --warmup  
100 --trials 3  
sudo python3 run.py cdf-db --work-dir /mnt/exp --ops 5000 --warmup  
100 --trials 3
```

Gambar IV.5: Perintah untuk menjalankan eksperimen ketiga beserta eksperimen keempat

IV.3.2 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan mekanisme yang dibangun benar-benar menghasilkan replika yang setara dengan sumbernya serta berlaku pada perangkat lunak nyata. Pengujian dibagi menjadi dua bagian.

IV.3.2.1 Pengujian Kesetaraan pada Setiap Titik Ukur

Verifikasi kesetaraan bukan merupakan langkah terpisah melainkan bagian dari setiap titik ukur sebagaimana disebutkan pada bagian IV.2.6. Setelah sinkronisasi selesai diukur, orkestrator eksperimen membandingkan direktori sumber dengan direktori replika menggunakan perbandingan berbasis *checksum* atas seluruh isi

kedua direktori. Hasil perbandingan tersebut dicatat pada berkas CSV sebagai kolom tersendiri bersama dengan penghitung kehilangan perekaman.

Pada seluruh 60 baris hasil eksperimen pertama beserta eksperimen kedua, ditambah 12 baris hasil eksperimen ketiga beserta eksperimen keempat, kedua kolom tersebut bernilai nol tanpa kecuali. Artinya tidak ada satu pun titik ukur yang menghasilkan replika berbeda dari sumbernya, sekaligus tidak ada satu pun titik ukur yang perekamannya dinyatakan tidak lengkap oleh mekanisme *fail-closed*. Hasil ini memenuhi kebutuhan F-9 sekaligus NF-1.

IV.3.2.2 Pengujian pada Perangkat Lunak Nyata

Kesetaraan yang tercapai pada beban kerja sintetis belum membuktikan mekanisme ini berlaku pada perangkat lunak nyata karena beban kerja sintetis hanya menempuh satu jalur tulis kernel. Oleh karena itu, pengujian dilakukan menggunakan tiga basis data yang berbeda dengan konfigurasi *default* masing-masing. Selanjutnya, diperiksa apakah kondisi basis data yang dijalankan menggunakan direktori pada mesin replika sama dengan kondisi basis data ketika dijalankan menggunakan direktori pada mesin utama. Langkah yang dilakukan pada setiap basis data adalah sebagai berikut.

1. Menjalankan aplikasi beserta basis datanya pada mesin utama dengan direktori data berada di dalam lingkup perekaman.
2. Mengirimkan sejumlah permintaan HTTP sehingga basis data menghasilkan perubahan pada direktori tersebut.
3. Menghentikan aplikasi sehingga seluruh perubahan selesai dituliskan.
4. Menerapkan artefak perubahan pada direktori data di mesin replika.
5. Menjalankan aplikasi yang sama pada mesin replika di atas direktori hasil penerapan tersebut.
6. Memeriksa bahwa aplikasi dapat berjalan secara normal dan data yang sebelumnya dituliskan dapat dibaca kembali dengan benar.

Hasil pengujian tersebut disajikan pada Tabel IV.3.

Tabel IV.3: Hasil pengujian pada aplikasi nyata

Basis data	Hasil pengujian	Keterangan Tambahan
SQLite	Lulus	Secara logis direktori replika dan direktori utama setara. Berkas indeks <i>write-ahead log</i> ditulis melalui pemetaan memori bersama sehingga tidak melewati titik penautan tulis. Berkas tersebut bersifat turunan yang dibangun ulang SQLite ketika basis data dibuka sehingga kesetaraan tidak terpengaruh
PostgreSQL	Lulus	Secara logis direktori replika dan direktori utama setara. Menuliskan <i>write-ahead log</i> menggunakan penulisan vektor. Segmennya dibuat sebagai berkas sementara lalu dinamai ulang ke tempatnya sehingga operasi penamaan ulang ikut menanggung beban
MySQL	Lulus	Secara logis direktori replika dan direktori utama setara. InnoDB menuliskan halaman berkas data melalui <i>AIO native</i> serta memperbesar <i>tablespace</i> menggunakan alokasi ruang berkas

Ketiga aplikasi tersebut berjalan tanpa modifikasi apa pun serta tanpa pemindahan direktori datanya. Yang membedakan ketiganya adalah jalur tulis kernel yang ditempuh masing-masing mesin penyimpanan, sebagaimana terlihat pada kolom keterangan Tabel IV.3. Kolom tersebut sekaligus menjelaskan mengapa titik penautan pada Tabel IV.2 tidak dapat dipersempit lebih jauh. Menghapus penautan pada penulisan vektor akan menyebabkan berkas *write-ahead log* PostgreSQL kosong pada replika, sedangkan menghapus penautan pada jalur asinkron akan menyebabkan tulisan InnoDB tidak terlihat sama sekali. Hasil pengujian ini menunjukkan mekanisme yang dibangun bersifat generik terhadap aplikasi sebagaimana *rsync*, sekaligus memenuhi kebutuhan NF-2.

IV.3.3 Analisis Hasil

Bagian ini menganalisis data dari keempat eksperimen. Penjelasan dimulai dari analisis biaya yang dibayar pada saat sinkronisasi, kemudian analisis biaya yang dibayar pada jalur eksekusi aplikasi, lalu ditutup dengan analisis keseluruhan.

IV.3.3.1 Konfigurasi Eksperimen

Keempat eksperimen dirancang untuk memisahkan dua tempat pembayaran biaya. Eksperimen pertama beserta eksperimen kedua mengukur biaya yang dibayar pada saat sinkronisasi, sedangkan eksperimen ketiga beserta eksperimen keempat meng-

ukur biaya yang dibayar pada jalur eksekusi aplikasi. Variabel yang dikunci beserta variabel yang divariasikan pada masing-masing eksperimen dapat dilihat pada Tabel IV.4.

Tabel IV.4: Variabel yang digunakan pada setiap eksperimen

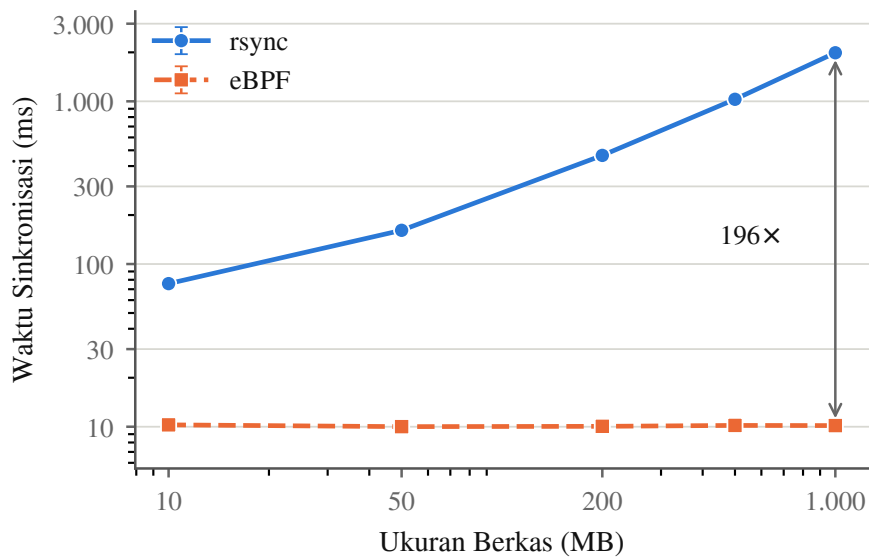
Eksperimen	Dikunci	Divariasikan	Yang ditunjukkan
Pertama	Perubahan 4 KB	Ukuran berkas 10 hingga 1000 MB	Biaya <i>rsync</i> ditentukan ukuran <i>state</i> , sedangkan program eBPF tidak.
Kedua	Satu berkas 10 MB	Banyaknya penulisan 1 hingga 10000	Situasi ketika program eBPF memiliki kinerja lebih buruk dibandingkan dengan <i>rsync</i> .
Ketiga	Penulisan 4 KB	Penangkapan mati atau hidup	Biaya yang ditimbulkan oleh program eBPF pada operasi <i>filesystem</i> secara langsung.
Keempat	Permintaan HTTP tulis	Penangkapan mati atau hidup	Biaya yang ditimbulkan oleh program eBPF pada aplikasi nyata tanpa modifikasi

Setiap titik ukur dijalankan sebanyak tiga kali. Nilai yang disajikan pada gambar merupakan rerata dari ketiga ulangan tersebut, sedangkan galat yang ditampilkan merupakan rentang antara nilai terkecil dengan nilai terbesar di antara ketiganya.

IV.3.3.2 Analisis Biaya akibat Ukuran Data

Pada eksperimen pertama, setiap operasi menghasilkan perubahan sebesar 4 KB. Ukuran berkas yang telah ada ditingkatkan dari 10 MB hingga 1000 MB, kemudian hasilnya dapat dilihat pada Gambar IV.6.

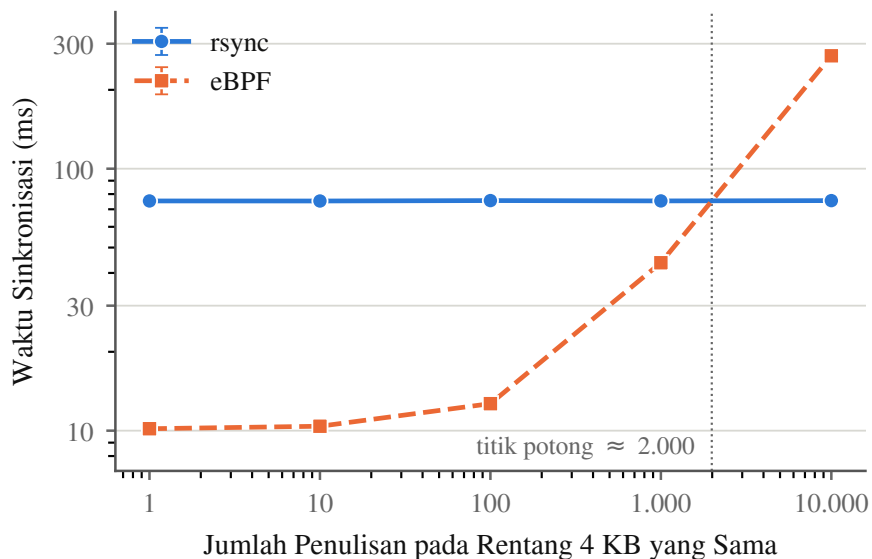
Biaya sinkronisasi pada *rsync* tumbuh dari 75,8 milidetik pada berkas berukuran 10 MB menjadi 1.988,2 milidetik pada berkas berukuran 1000 MB. Pertumbuhan tersebut sebanding dengan ukuran berkas karena kedua sisi harus membaca berkas secara utuh untuk menentukan blok mana yang berbeda. Biaya penangkapan sebaliknya tidak bergerak sama sekali, yaitu berkisar antara 10,0 milidetik hingga 10,3 milidetik pada seluruh titik ukur. Isi perubahan data sudah dipegang oleh program penangkap eBPF sejak operasi terjadi sehingga tidak ada pembacaan ulang berkas sumber yang perlu dilakukan. Rasio antara keduanya tumbuh dari 7,4 kali pada berkas 10 MB menjadi 195,6 kali pada berkas 1000 MB.



Gambar IV.6: Biaya menghasilkan satu artefak perubahan terhadap ukuran berkas yang sudah ada, dengan volume perubahan tetap sebesar 4 KB

IV.3.3.3 Analisis Biaya terhadap Penulisan Berulang

Eksperimen kedua menguji pengaruh frekuensi perubahan terhadap waktu sinkronisasi. Ukuran berkas ditetapkan sebesar 10 MB, kemudian rentang *byte* berukuran 4 KB ditulis ulang sebanyak satu hingga sepuluh ribu kali sebelum sinkronisasi dijalankan. Meskipun jumlah penulisan berbeda, ukuran perubahan akhir antara sumber dan replika tetap sebesar 4 KB karena setiap penulisan berikutnya menimpa hasil penulisan sebelumnya. Hasil eksperimen dapat dilihat pada Gambar IV.7.



Gambar IV.7: Biaya sinkronisasi ketika rentang *byte* 4 KB yang sama ditulis ulang sebelum sinkronisasi dijalankan

Biaya *rsync* dalam bentuk waktu relatif konstan, yaitu berada pada kisaran 75,3

hingga 75,6 milidetik di seluruh titik pengukuran. Hal ini disebabkan oleh cara kerja *rsync* yang membandingkan kondisi akhir direktori, sehingga jumlah penulisan yang dilakukan untuk mencapai kondisi tersebut tidak memengaruhi biaya. Sebaliknya, biaya penangkapan dalam bentuk waktu meningkat dari 10,2 milidetik pada satu penulisan menjadi 269,8 milidetik pada sepuluh ribu penulisan. Hal ini terjadi karena program eBPF menyimpan setiap operasi secara terpisah tanpa melakukan penggabungan, sehingga ukuran artefak bertambah secara linier seiring dengan jumlah penulisan. Kedua kurva berpotongan pada sekitar dua ribu penulisan.

Hasil ini merupakan batas keberlakuan mekanisme yang dibangun. Menangkap perubahan pada saat operasi berlangsung berarti menangkap setiap operasi, termasuk operasi yang efeknya kemudian ditimpa oleh operasi berikutnya. Pendekatan berbasis pemindaian tidak menghadapi persoalan tersebut karena ia hanya melihat hasil akhirnya.

IV.3.3.4 Analisis Biaya pada Jalur Eksekusi Aplikasi

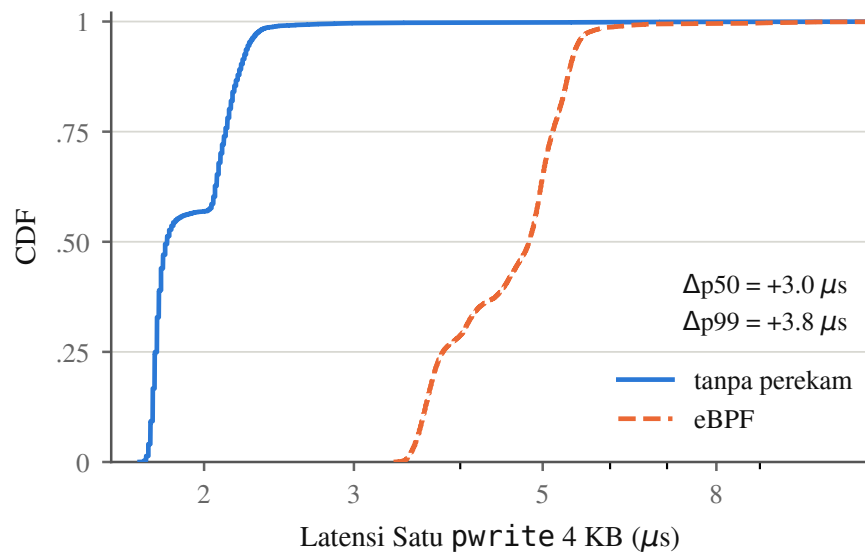
Kedua eksperimen sebelumnya mengukur biaya yang dibayar pada saat sinkronisasi. Bagian ini mengukur biaya yang dibayar oleh eBPF karena hidup pada jalur eksekusi. Pengukuran dilakukan dalam dua kondisi, yaitu ketika program eBPF diaktifkan dan dinonaktifkan. Setiap kondisi menghasilkan 15.000 sampel yang diperoleh dari tiga percobaan, dengan masing-masing percobaan menjalankan 5.000 operasi.

Kondisi ketika eBPF dinonaktifkan dapat merepresentasikan mekanisme *rsync* karena *rsync* tidak menimbulkan biaya tambahan pada jalur eksekusi aplikasi. Dengan demikian, biaya yang diukur pada kondisi tersebut merepresentasikan biaya operasi aplikasi tanpa tambahan dari mekanisme sinkronisasi. Biaya *rsync* sendiri muncul saat proses sinkronisasi, yang telah diukur pada Bagian IV.3.3.2 dan IV.3.3.3.

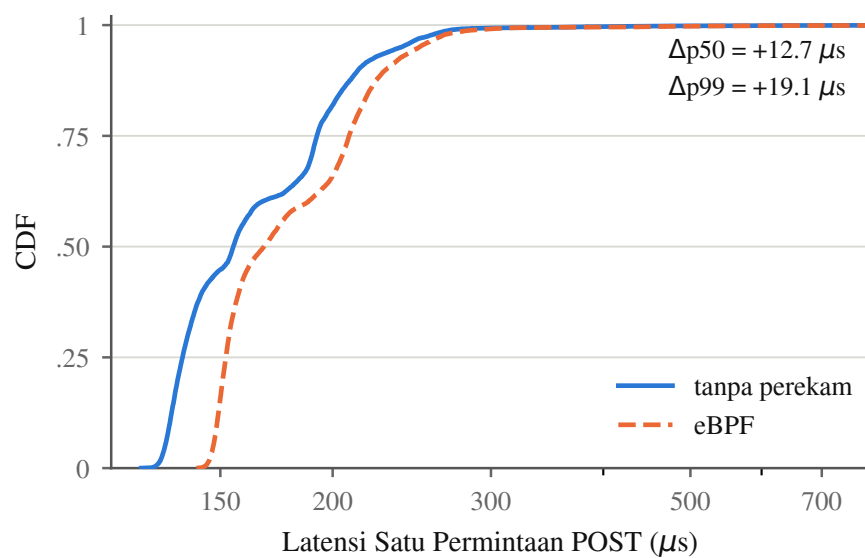
Hasil pengukuran pada operasi *filesystem* disajikan pada Gambar IV.8.

Pada kondisi tanpa perekam, persentil ke-50, ke-90, serta ke-99 masing-masing sebesar 1,81, 2,21, serta 2,45 mikrodetik. Ketika perekam diaktifkan, nilainya meningkat menjadi 4,82, 5,39, serta 6,24 mikrodetik, sehingga biaya tambahannya berturut-turut sebesar 3,0, 3,2, serta 3,8 mikrodetik.

Hasil pengukuran pada aplikasi nyata di atas SQLite disajikan pada Gambar IV.9.



Gambar IV.8: Distribusi latensi satu penulisan berukuran 4 KB dengan penangkapan mati dibandingkan hidup



Gambar IV.9: Distribusi latensi satu permintaan HTTP tulis pada aplikasi daftar tugas di atas SQLite dengan penangkapan mati dibandingkan hidup

Pada kondisi tanpa perekam, persentil ke-50, ke-90, serta ke-99 masing-masing sebesar 155,1, 213,8, serta 273,2 mikrodetik. Dengan perekam, nilainya menjadi 167,8, 229,4, serta 292,2 mikrodetik, sehingga biaya tambahannya sebesar 12,7, 15,6, serta 19,1 mikrodetik.

berdasarkan eksperimen, penangkap eBPF terlihat memberi biaya tambahan pada aplikasi yang direplikasi. Biaya tersebut relatif wajar karena penangkap berjalan pada jalur eksekusi aplikasi sehingga setiap operasi tulis harus menunggu penyalinan isi perubahan selesai sebelum dapat dilanjutkan. Meskipun demikian, besarnya tetap terbatas pada orde mikrodetik sehingga pengaruhnya terhadap latensi aplikasi tergolong kecil.

IV.3.3.5 Analisis Keseluruhan

Setelah menganalisis keempat eksperimen, bagian ini merangkum temuan yang diperoleh. Fokusnya adalah membandingkan program penangkap perubahan berbasis eBPF dengan program penangkap perubahan berbasis pemindaian.

Kedua mekanisme memiliki biaya dengan besaran yang sebanding, tetapi biaya tersebut muncul pada bagian yang berbeda. Program berbasis pemindaian membayar biaya saat sinkronisasi, yang dipengaruhi oleh ukuran *state* yang harus dipindai. Sebaliknya, program berbasis eBPF membayar biaya saat aplikasi melakukan penulisan, yang dipengaruhi oleh jumlah operasi yang terjadi. Karena keduanya dipengaruhi oleh faktor yang berbeda, keunggulan masing-masing bergantung pada karakteristik beban kerja.

- Program penangkap perubahan berbasis eBPF unggul ketika *state* berukuran besar, tetapi perubahan yang terjadi kecil dan jarang. Pada kondisi tersebut, *rsync* harus membaca seluruh berkas untuk menemukan perubahan yang hanya berukuran beberapa KB. Sementara itu, program berbasis eBPF sudah memperoleh isi perubahan tersebut ketika operasi berlangsung. Biaya yang harus dibayar adalah tambahan beberapa mikrodetik pada setiap operasi tulis aplikasi.
- *rsync* unggul ketika rentang *byte* yang sama ditulis berulang kali sebelum sinkronisasi dilakukan. Pada kondisi tersebut, program berbasis eBPF menyimpan setiap operasi tanpa melakukan penggabungan sehingga ukuran ar-

tefaknya dapat melebihi ukuran perubahan sebenarnya. Sebaliknya, program berbasis pemindaian hanya melihat kondisi akhir sehingga biayanya tetap.

Pemilihan antara kedua mekanisme tersebut perlu mempertimbangkan tiga hal, yaitu ukuran *state* yang direplikasi, frekuensi penulisan ulang pada rentang *byte* yang sama, dan tambahan latensi pada jalur tulis yang dapat ditoleransi aplikasi. Untuk beban kerja dengan *state* besar yang mengalami sedikit perubahan, program berbasis eBPF memberikan biaya sinkronisasi yang tidak bergantung pada ukuran *state*. Sebaliknya, untuk beban kerja yang menulis berulang kali pada lokasi yang sama dengan interval sinkronisasi yang panjang, program berbasis pemindaian lebih menguntungkan.

Pengujian pada aplikasi nyata di bagian IV.3.2.2 menunjukkan bahwa keunggulan tersebut tidak dibayar dengan hilangnya sifat generik. Mekanisme yang dibangun bekerja pada tiga mesin penyimpanan yang berbeda tanpa modifikasi aplikasi maupun pemindahan direktori data, meskipun ketiganya menempuh jalur tulis kernel yang berbeda-beda.

BAB V

PENUTUP

Bab ini memaparkan kesimpulan dari rancangan dan evaluasi mekanisme sinkronisasi direktori inkremental berbasis eBPF. Bab ini juga menyampaikan saran untuk pengembangan mekanisme pada penelitian selanjutnya.

V.1 Kesimpulan

Penelitian ini merancang dan mengevaluasi mekanisme sinkronisasi direktori inkremental yang menangkap operasi pada lapisan *Virtual File System* menggunakan eBPF. Berdasarkan hasil yang diperoleh, terdapat tiga kesimpulan.

1. Mekanisme yang dibangun dapat menangkap operasi mutasi pada direktori sumber serta menyusunnya menjadi artefak perubahan untuk diterapkan pada direktori replika di mesin lain. Penentuan perubahan oleh mekanisme ini tidak memerlukan pemindaian ulang pohon direktori maupun pembacaan ulang isi berkas sumber.
2. Seluruh titik ukur sinkronisasi yang diverifikasi menghasilkan direktori replika yang setara dengan direktori sumber. Mekanisme juga berhasil diterapkan pada SQLite, PostgreSQL serta MySQL tanpa modifikasi aplikasi maupun pemindahan direktori data.
3. Untuk perubahan sebesar 4 KB, waktu sinkronisasi *rsync* meningkat dari 75,8 milidetik pada berkas 10 MB menjadi 1.988,2 milidetik pada berkas 1000 MB. Biaya mekanisme eBPF berada pada kisaran 10,0 hingga 10,3 milidetik. Namun, penulisan berulang pada rentang *byte* yang sama meningkatkan biaya mekanisme eBPF dan membuat *rsync* lebih menguntungkan setelah sekitar dua ribu penulisan. Penangkapan eBPF juga menambah latensi operasi tulis pada orde mikrodetik sehingga penggunaannya perlu mempertimbangkan karakteristik beban kerja serta tambahan latensi yang dapat ditoleransi aplikasi.

V.2 Saran

Penelitian ini masih memiliki beberapa batasan yang dapat dikembangkan pada penelitian selanjutnya. Pengembangan yang dapat dilakukan adalah sebagai berikut.

1. Menambahkan penggabungan atau pemadatan operasi yang menulis ulang rentang *byte* yang sama. Pengembangan ini dapat mengurangi pertumbuhan artefak perubahan pada beban kerja dengan penulisan berulang.
2. Memperluas dukungan terhadap metadata serta objek *filesystem* yang belum dicakup, seperti *symbolic link*, *hard link*, atribut tambahan serta ACL. Jalur I/O yang belum teramati juga perlu dievaluasi.
3. Mengevaluasi mekanisme pada variasi versi kernel, jenis *filesystem* serta kondisi jaringan yang lebih beragam. Pengujian pemulihan setelah kegagalan mesin juga diperlukan untuk menilai pemakaian mekanisme pada skenario cadangan nyata.

DAFTAR PUSTAKA

- Btrfs developers. (2026). *Send/receive* [Diakses 26 Agustus 2026]. <https://btrfs.readthedocs.io/en/stable/Send-receive.html>
- IBM. (2025). What Is Data Synchronization? [Diakses 30 November 2025]. <https://www.ibm.com/think/topics/data-synchronization>
- LINBIT. (2020). DRBD 9.0 User's Guide [Diakses 12 November 2025]. https://linbit.com/drbd-user-guide/drbd-guide-9_0-en/
- Linux Foundation. (2015). *Filesystem Hierarchy Standard (FHS) 3.0* [Diakses 20 Desember 2025]. https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html
- Linux kernel developers. (2026a). *Overview of the Linux Virtual File System* [Diakses 26 Agustus 2026]. <https://docs.kernel.org/filesystems/vfs.html>
- Linux kernel developers. (2026b). *Program Types and ELF Sections* [Diakses 26 Agustus 2026]. https://docs.kernel.org/bpf/libbpf/program_types.html
- Linux Kernel Teaching. (2025). *File System Implementation* [Diakses 21 Desember 2025]. <https://linux-kernel-labs.github.io/refs/heads/master/so2/lec8-filesystems.html>
- Linux man-pages project. (2026). *inotify(7) — Linux manual page* [Diakses 26 Agustus 2026]. <https://man7.org/linux/man-pages/man7/inotify.7.html>
- Oracle Corporation. (2025). *MySQL 8.4 Reference Manual: Binary Log File Position Based Replication Configuration Overview* [Diakses 21 Desember 2025]. <https://dev.mysql.com/doc/refman/8.4/en/binlog-replication-configuration-overview.html>
- PostgreSQL Global Development Group. (2025a). *PostgreSQL Documentation: Creating a Database Cluster* [Diakses 20 Desember 2025]. <https://www.postgresql.org/docs/current/creating-cluster.html>
- PostgreSQL Global Development Group. (2025b). *PostgreSQL Documentation: Log-Shipping Standby Servers* [Diakses 21 Desember 2025]. <https://www.postgresql.org/docs/current/warm-standby.html>

- rsync contributors. (2026). *rsync(1) – a fast, versatile, remote (and local) file-copying tool* [Diakses 26 Agustus 2026]. <https://rsync.samba.org/ftp/rsync/rsync.1>
- Sharaf, H., Ahmad, I., & Dimitriou, T. (2022). Extended Berkeley Packet Filter: An Application Perspective. *IEEE Access*, *10*, 126370–126393. <https://doi.org/10.1109/ACCESS.2022.3226269>
- Tridgell, A., & Mackerras, P. (1996). *The rsync algorithm* (techreport TR-CS-96-05) [PDF tersedia di [paper/tridgell96.pdf](https://www.cse.uq.edu.au/~tridgell/paper/tridgell96.pdf)]. Australian National University.
- Witzke, J., L"oßer, A., Bountris, V., Schintke, F., & Scheuermann, B. (2024). Low-level I/O Monitoring for Scientific Workflows. <https://arxiv.org/abs/2408.00411>